AI-Powered Prompt Enhancement Chrome Extension

Project report

A Manifest V3 Chrome extension that rewrites the prompt in the focused text box into a clearer, better-specified prompt using the user's own Groq API key. It applies task-dependent prompt-engineering techniques, validates every result locally, replaces the text in place with full undo, and never submits anything on the user's behalf.

Item	Value
Report generated	2026-11-02
Project version / milestone	v0.1.0 — Groq MVP (milestones M0–M4 complete; M6/M7 partial; M5 Gemini deferred)
Repository / branch	KernelLex/prompt-ext · claude/eloquent-darwin-i4nfne
Prepared by	Claude Code (AI coding assistant), from the project conversation and repository artifacts
Evidence policy	Screenshots are genuine captures of the running extension unless labelled CONCEPTUAL MOCKUP. Provider responses in screenshots come from a local mock server; the live Groq API was unreachable from the build environment.

Contents

2. Executive summary 4

3. Original user requirements 5

3.1 Message 1 — Initial extension concept, plan.md-first workflow, Groq and Gemini requirements 5

3.2 Message 2 — Request for prototype screenshots (after the Phase 1 plan) 12

3.3 Message 3 — Decisions resolving the open questions; approval to implement 12

3.4 Message 4 — API key (redacted) and the requirements for this report 13

4. AI response excerpts 16

4.1 Response to message 1 — Phase 1 discovery and plan.md 16

4.2 Response to message 2 — prototype screenshots 16

4.3 Response to message 3 — decisions and implementation 16

4.4 Response to message 4 — API key and report 17

5. Planning and architecture 18

5.1 Summary of plan.md 18

5.2 System architecture 18

5.3 Components 19

5.4 Enhancement pipeline 19

5.5 Provider interface 19

5.6 Major design decisions and alternatives 19

6. Prompt engineering methodology 21

6.1 Frameworks as checklists, not templates 21

6.2 How selection depends on task type and mode 21

6.3 Before-and-after examples 21

6.4 How quality is evaluated 22

7. Implementation summary 23

7.1 Implemented features 23

7.2 Technologies, dependencies and structure 23

7.3 Groq integration (and Gemini status) 23

7.4 API keys, permissions and data 24

7.5 Planned but not implemented 24

8. Prototype screenshots and visual evidence 25

8.1 Core journey: before, enhancing, after, undo 25

8.2 Safety and error states 27

8.3 Settings and installation in Chromium 30

8.4 Conceptual mockups from the planning phase 33

9. Testing and validation 35

9.1 Executed — results 35

9.2 Unit test coverage by area . . . 35
9.3 Browser tests (36) . . . 35
9.4 Test validity check . . . 36
9.5 Planned but not executed . . . 36
10. Challenges and decisions . . . 37
10.1 Technical challenges and solutions . . . 37
10.2 Trade-offs . . . 37
10.3 Unresolved risks and known limitations . . . 37
11. Installation and usage guide . . . 38
11.1 Build and load . . . 38
11.2 Configure a Groq key securely . . . 38
11.3 Use . . . 38
11.4 Troubleshooting . . . 38
11.5 Supported editors . . . 38
12. Conclusion and future roadmap . . . 39
12.1 What was achieved . . . 39
12.2 Remaining limitations . . . 39
12.3 Prioritized next steps . . . 39
12.4 Deliverable locations . . . 39

2. Executive summary

Problem

People write prompts quickly and vaguely. Under-specified prompts produce generic or wrong answers, while generic 'prompt improvers' often bloat short questions into long templates, invent requirements, or require copy-pasting into a separate tool.

Solution

An in-place enhancer: the user presses **Alt+Shift+E** (or the toolbar icon) inside any supported text box. The extension reads only that box, makes **one** request to Groq with a strict rewriting specification, validates the result locally (preserved numbers, quotes, code and URLs; no answering instead of rewriting; no language switch), and replaces the text in the same box. Undo is one click or **Alt+Shift+Z**. The user always decides whether to send.

Implemented features (v0.1.0)

- Enhancement engine: input validation, task-type hint, framework registry (CO-STAR, RISEN, RTF, CREATE, APE + 12 techniques), Light/Balanced/Deep modes, single-call request builder, shared response contract, deterministic output validator.
- Groq provider adapter behind a provider-neutral interface, with error taxonomy, ≤ 1 bounded retry, quota circuit breaker and key redaction.
- Editor integration: textarea, text input, React-controlled textarea, contenteditable, open shadow roots, ProseMirror; verified insertion with rollback; field-changed protection; undo/restore; cancel.
- Options page: key save/test/remove, model choice, mode, explicit data-use consent, session-only key option, opt-in 'Insert anyway'.
- Minimal permissions: activeTab, scripting, storage + optional api.groq.com host permission. No telemetry.
- 149 automated tests executed and passing (113 unit, 36 browser tests with the real unpacked extension in Chromium).

Technologies and providers

TypeScript (strict), Chrome Manifest V3, vanilla DOM UI, esbuild, Vitest, Playwright, ESLint. AI provider: **Groq** (OpenAI-compatible chat completions API; default model `openai/gpt-oss-120b`). **Google Gemini was deferred** by the user's decision D2 and is not implemented.

Honest status

All automated tests pass against a **local mock** of Groq's API. **No request was sent to the real Groq API**: the build environment's network policy blocks `api.groq.com` (HTTP 403 at the proxy). Real-site testing on ChatGPT/Claude.ai and human quality scoring were not performed. These gaps are listed in Sections 9, 10 and 12.

3. Original user requirements

The four user messages below are reproduced **verbatim**, in chronological order, from the project conversation (the complete conversation history was available in the session). Only one change was made: the API key in message 4 is replaced by a redaction notice. Message 3 is reproduced as received, including its truncated final line. Message 4 arrived while message 3 was being implemented.

3.1 Message 1 — Initial extension concept, plan.md-first workflow, Groq and Gemini requirements

PROJECT: AI-Powered Prompt Enhancement Chrome Extension
1. ROLE — WHO YOU ARE
Act as a senior software architect, Chrome Extension engineer, AI integration specialist, prompt engineering expert, UX designer, and QA engineer.
Your responsibility is to design and build a reliable, secure, maintainable Chrome extension that improves user-written prompts using established prompt engineering principles and frameworks.
Think critically. Challenge weak assumptions, investigate technical limitations, and justify architectural decisions.
Do not build features merely because they sound impressive.
Work in clearly defined phases with measurable deliverables.
Your first task is to inspect the repository and create `plan.md`. Do not implement the extension until I have reviewed and explicitly approved the plan.
Do not skip this requirement, even if you believe the implementation is straightforward.
2. CONTEXT — WHAT WE ARE BUILDING
I want a Chrome extension that improves prompts directly inside editable text fields on websites.
The core experience should be:

1. The user types a prompt into a text field on a supported website.
2. The user activates the extension using a toolbar button, keyboard shortcut, or another approved interaction.
3. The extension captures the prompt from the active editable field.
4. An enhancement engine analyzes the prompt, identifies the task, and chooses suitable prompt engineering techniques.
5. The engine rewrites the prompt to improve clarity, structure, specificity, and expected output quality.
6. The enhanced prompt replaces the original text in the same field.
7. The user can inspect the result, undo the replacement, and restore the original prompt.
8. The user decides whether to submit the enhanced prompt.

The extension must never submit the prompt automatically.
The user should not need to open a separate dashboard or manually copy and paste prompts between applications.
The product should work across multiple supported websites, including AI chat interfaces, coding tools, writing applications, and research tools.
Do not promise compatibility with every website. Define a realistic support matrix and test actual editor behavior.
Core product philosophy
The extension should optimize for prompt effectiveness, not prompt length.
A short question should remain short when it is already effective. A complex task should receive more structure when that structure meaningfully improves the instructions.
The extension must preserve the user's intent, facts, terminology, constraints, and desired outcome.
It must not fabricate missing context, introduce arbitrary requirements, or change the user's objective.
3. OBJECTIVES — PRODUCT REQUIREMENTS
3.1 Prompt capture and replacement
The extension must:

* Detect the active editable field when enhancement is requested.
* Read its current text accurately.
* Preserve the original prompt before making any changes.
* Send only the necessary prompt content and explicitly approved context to the selected AI provider.
* Insert the enhanced prompt into the same field.
* Trigger the appropriate input events for supported websites.
* Preserve focus wherever possible.
* Avoid duplicate enhancement requests from repeated clicks.
* Allow the user to cancel or recover from failed requests where practical.
* Provide reliable undo and restoration of the original text.
* Never submit the prompt, click Send, or execute the user's instructions.

Investigate support for:

* HTML `textarea` elements.
* Text inputs.
* `contenteditable` elements.
* Rich-text editors.
* Framework-controlled inputs, including React-based applications.
* Shadow DOM where accessible.
* Iframes and cross-origin restrictions.
* Websites that replace their editor dynamically.

Use safe, supported techniques. Do not depend on brittle DOM manipulation or framework internals when a more reliable integration is available.
If an editor cannot be updated safely, explain the limitation and provide a non-destructive fallback where feasible.
3.2 Prompt enhancement engine
Design a modular enhancement pipeline with the following stages:

1. Input validation: Determine whether the active field contains a usable prompt.
2. Intent analysis: Identify what the user is trying to accomplish.
3. Task classification: Classify the task into categories such as coding, research, writing, analysis, factual question, planning, or creative work.
4. Quality analysis: Identify ambiguity, missing constraints, vague instructions, insufficient context, and unspecified output formats.
5. Framework selection: Select the most suitable framework or combination of techniques.
6. Prompt rewriting: Produce a clear, actionable, well-structured prompt.
7. Validation: Check that the result preserves intent and does not introduce unsupported facts or requirements.
8. Presentation: Return the enhanced prompt in a predictable format.
9. Replacement: Update the active editor only after a successful response and appropriate validation.

Keep these responsibilities logically separate.
Avoid unnecessary model calls. Determine which processing stages can be combined into a single request without sacrificing quality or maintainability.
3.3 Prompt engineering frameworks
The engine should understand and selectively apply the following frameworks and techniques:

* CO-STAR: Context, Objective, Style, Tone, Audience, Response.
* RISEN: Role, Instructions, Steps, End Goal, Narrowing.
* RTF: Role, Task, Format.
* CREATE: Context, Role, Explicit, Action, Target Audience, Exclusions.
* APE: Action, Purpose, Expectation.
* Role and perspective specification.
* Explicit instructions and step sequencing.
* Task decomposition.
* Context enrichment.
* Output format and schema specification.
* Few-shot examples when useful.
* Explicit success criteria.
* Scope boundaries and exclusions.
* Ambiguity reduction.
* Source-material delimitation.
* Appropriate handling of untrusted input.

Treat framework definitions as conventions that should be verified. Do not assume acronyms have universally standardized meanings.
Framework selection must be task-dependent. The engine must not mechanically force every prompt into CO-STAR or RISEN.
Examples:

* A simple factual question may need only clearer wording.
* A coding request may benefit from language, environment, constraints, expected behavior, and test requirements.
* A research request may need scope, evidence requirements, source expectations, and a structured output.
* A creative-writing request may benefit from audience, tone, style, and creative constraints.
* An analytical task may benefit from explicit evaluation criteria and output structure.

Create a framework registry or equivalent modular architecture so new frameworks and strategies can be added without rewriting the entire enhancement engine.
For each framework, define its intended use, selection criteria, instructions, limitations, and representative tests.
3.4 Enhancement modes
Plan for three levels of enhancement:

* Light: Correct ambiguity and improve clarity with minimal changes.
* Balanced: Improve instructions, context, constraints, and output format where useful.
* Deep: Apply more comprehensive task analysis, structure, decomposition, and success criteria when justified.

Balanced should be the default unless the technical evaluation supports another choice.
The selected mode should affect the enhancement strategy, not simply the length of the generated prompt.
Consider task-specific presets such as:

* General.
* Coding.
* Research.
* Writing.
* Analysis.
* Creative.

Keep these optional presets separate from the core enhancement engine.
For the MVP, implement only the settings necessary to validate the central experience. Do not overbuild the interface.
4. AI PROVIDERS — GROQ AND GOOGLE GEMINI
The initial AI providers must be:

1. Groq API: https://console.groq.com/
2. Google Gemini API: https://aistudio.google.com/

Both providers must be supported through a provider-neutral architecture.
The initial prototype should prioritize access through their available free API tiers, where eligible. Do not assume free access means unlimited usage, permanent availability, or permission for every deployment scenario.
4.1 Research before deciding
Before choosing the first provider, verify current official documentation for both services.
Investigate:

* Current available models.
* Free-tier eligibility and restrictions.
* Requests-per-minute and requests-per-day limits where applicable.
* Token and context limits.
* Model suitability for prompt rewriting and instruction following.
* Latency and response quality.
* API authentication.
* Supported request and response formats.
* Rate-limit and quota-exhaustion errors.
* Privacy and data-handling terms.
* Commercial-use and distribution restrictions.
* Model deprecation and availability policies.

Include a comparison table in `plan.md`.
Recommend which provider should be implemented first based on the actual requirements and verified evidence.
Do not select a provider solely because it advertises a free tier.
Record the documentation links and the date of verification.
4.2 Provider-neutral architecture
Create a shared provider interface so Groq and Gemini can be used interchangeably.
Separate:

* Enhancement pipeline.
* Framework selection.
* Shared enhancement instructions.
* Provider adapters.
* Request construction.
* Response parsing.
* Configuration.
* Credential management.
* Error handling.
* Response validation.

Do not scatter provider-specific conditionals throughout the application.
Provider-specific models, endpoints, and configuration should be centralized.
The user should be able to select a configured provider through the settings interface.
A future provider should be addable without rewriting the core enhancement engine.
4.3 API key management
For the personal-use prototype, support user-supplied API keys for Groq and Gemini.
The settings interface should allow the user to:

* Select a provider.
* Enter or update that provider's API key.
* Select from verified, supported models.
* Test provider connectivity where feasible.
* View understandable configuration and authentication errors.
* Remove a saved key.

Security requirements:

* Never hardcode real API keys into source code.
* Never commit real credentials to Git.
* Do not expose keys in URLs, logs, telemetry, error messages, or debugging output.
* Never treat a bundled `.env` file as secure once its contents are included in an extension build.
* Do not claim that Chrome extension storage makes provider keys inaccessible to malicious extensions or local users.
* Explain the security implications of saving API keys in browser extension storage.
* Request only necessary Chrome permissions.
* Avoid collecting or transmitting data unrelated to the enhancement request.

For the initial personal-use prototype, direct provider access using the user's own key is acceptable if the risks are documented and the architecture is designed appropriately.
For a public release, evaluate a backend proxy that keeps provider credentials off the client. Consider authentication, hosting expenses, rate limiting, abuse prevention, quotas, and operational maintenance.
Do not introduce a backend automatically. Recommend the simplest viable approach for the intended deployment.
4.4 Provider fallback and cost control
Allow the user to configure a default provider and, if practical, a manual alternative.
Do not silently send a user's prompt to another provider after a failure.
Any automatic provider fallback must be explicitly enabled, clearly explained, and limited to the configured providers.
Implement reasonable safeguards against:

* Infinite retries.
* Duplicate requests.
* Unnecessary model calls.
* Repeated quota failures.
* Excessive request sizes.
* Unexpected API usage.

Use bounded retries only when appropriate. Respect provider rate-limit guidance and distinguish transient failures from authentication or quota failures.
4.5 Consistent output
Both providers must follow the same core enhancement specification.
Use a predictable response contract that separates:

* Enhanced prompt text.
* Optional framework or task metadata.
* Validation status, if needed internally.

Do not expose provider-specific formatting to the user.
Validate responses before inserting them into an editor. If the response is empty, malformed, or otherwise unusable, preserve the original prompt and display an error.
Do not automatically replace the user's prompt with an unvalidated response.
5. UX AND INTERACTION DESIGN
The interface must be simple, fast, and unobtrusive.
Evaluate the following MVP capabilities:

* Toolbar action to enhance the active prompt.
* Keyboard shortcut.
* Clear progress state.
* Success and failure feedback.
* Undo and restore original prompt.
* Provider and API key settings.
* Enhancement mode selection.
* Minimal first-use instructions.

Consider, but do not automatically include, these future capabilities:

* Original-versus-enhanced comparison.
* Framework explanations.
* Prompt history.
* Custom enhancement instructions.
* Domain-specific presets.
* User-defined frameworks.
* Local prompt templates.
* Enhancement quality feedback.

Prompt history should be opt-in if implemented. Define retention and deletion behavior before storing prompts.
Interaction rules

* The extension must not interfere with normal typing when inactive.
* Enhancement must require an intentional user action.
* Do not automatically enhance every keystroke.
* Do not submit the prompt.
* Preserve the original text until the replacement is confirmed.

* Clearly communicate when the selected editor is unsupported.
* Prevent duplicate requests while an enhancement is already in progress.
* Ensure that the user can recover the original prompt if the enhancement fails.
* Make keyboard shortcuts configurable if supported and justified.

The extension must remain useful without requiring users to understand prompt engineering frameworks.
6. SECURITY, PRIVACY, AND RELIABILITY
Use Chrome Extension Manifest V3.
Investigate the appropriate division of responsibilities between:

* Popup or extension UI.
* Content scripts.
* Background service worker.
* Storage.
* Provider adapters.
* Optional backend.

Do not give content scripts or the service worker unnecessary responsibilities or permissions.
Required safeguards

1. Request only the permissions required by the selected architecture.
2. Prefer narrow host permissions where feasible.
3. Avoid broad access to all websites unless the design genuinely requires it and the permissions are clearly justified.
4. Never transmit page content unrelated to the active prompt.
5. Do not transmit the full page or conversation history by default.
6. Treat website content and user-provided material as untrusted input.
7. Do not let prompt text override application-level security rules or cause unintended tool execution.
8. Do not execute model-generated code or scripts.
9. Keep API credentials out of logs and distributed source.
10. Avoid unnecessary analytics and telemetry.
11. Document what information is transmitted to Groq or Gemini.
12. Provide appropriate user disclosure before external processing.
13. Handle provider failures without destroying the user's original text.
14. Do not claim that extension-side prompt instructions provide a security boundary against a malicious model response.

Distinguish the security of the application's own behavior from prompt injection risks within text being rewritten.
Document any remaining risks that cannot be eliminated by the proposed architecture.
7. TECHNICAL APPROACH
Evaluate a maintainable technology stack, considering:

* TypeScript.
* Chrome Extension Manifest V3.
* A lightweight UI framework or vanilla TypeScript.
* A suitable build tool.
* A test runner.
* Linting and type checking.
* Chrome extension development and debugging workflows.

Choose the simplest architecture that satisfies the requirements.
Do not add React, a backend framework, a database, or additional dependencies simply because they are popular.
Explain the trade-offs in `plan.md`.
The implementation should use modular code and clear interfaces. Avoid enormous files, unnecessary abstractions, duplicated logic, and hard-to-test dependencies.
Follow the repository's existing conventions if a project already exists and those conventions are reasonable.
Do not overwrite existing user work.
8. PROMPT QUALITY EVALUATION
A successful enhancement is not necessarily a longer or more elaborate prompt.
Design a test dataset containing representative examples such as:

* A simple factual question.
* An underspecified coding task.
* A research request.
* A professional writing request.
* A creative task.
* A complex analytical task.
* A prompt with explicit constraints that must be preserved.
* A prompt that is already well-written.
* A prompt containing insufficient information.
* A prompt containing adversarial or untrusted quoted text.

Evaluate the following dimensions:

1. Intent preservation.
2. Factual and constraint preservation.
3. Clarity.
4. Specificity.
5. Relevance of added structure.
6. Output-format clarity.
7. Conciseness.
8. Absence of invented requirements.
9. Framework-selection appropriateness.
10. Failure safety.

Define measurable acceptance criteria and compare Groq and Gemini against the same dataset.
Include regression tests so changes to the enhancement instructions do not silently degrade prompt quality.
Use a combination of deterministic assertions and documented human evaluation. Do not pretend a subjective prompt-quality score is an objective guarantee of model performance.
Set a reasonable response-time target for the MVP and explain how it will be measured.
9. REQUIRED DEVELOPMENT WORKFLOW
You must execute this project in three phases.
PHASE 1 — DISCOVERY AND `plan.md`
This is the only phase you are authorized to execute initially.
Start by inspecting the current working directory and repository.
Determine:

* Whether the repository is empty or already contains a project.
* Which files and configurations already exist.
* Whether any implementation or tests already exist.
* Which technical constraints affect the proposed architecture.

Consult relevant official documentation to resolve important technical uncertainties.
Then create a comprehensive `plan.md` at the project root.
If `plan.md` already exists, inspect it first and preserve useful content.
Required `plan.md` structure
The document must contain the following sections.

1. Executive summary: Product purpose and proposed MVP.
2. Goals and non-goals: Explicit scope boundaries.
3. User stories: Primary workflows and expected behavior.
4. Functional requirements: Observable behavior of each MVP feature.
5. Non-functional requirements: Reliability, security, performance, accessibility, and maintainability.
6. Architecture: Components, responsibilities, and communication.
7. Architecture diagram: Mermaid diagram showing data flow from the active editor through the extension and selected provider back to the editor.
8. Technology choices: Candidate stacks, trade-offs, and recommended decisions.
9. AI provider comparison: Groq versus Gemini, supported models, verified free-tier constraints, documentation references, and recommended starting provider.
10. Credential architecture: Direct API access versus backend proxy, with risks and costs.
11. Enhancement pipeline: Input analysis, classification, framework selection, rewriting, validation, and replacement.
12. Framework registry: Framework definitions, selection criteria, modular interfaces, and tests.
13. UX design: Toolbar action, keyboard shortcut, progress states, settings, undo, and errors.
14. Editor compatibility matrix: Supported elements, known limitations, and fallback behavior.
15. Security and privacy: Permissions, API keys, prompt transmission, retention, and threat model.
16. Data model: Settings and any necessary persistent state.
17. Project structure: Proposed directory and file layout.
18. Implementation milestones: Ordered tasks, dependencies, and completion criteria.
19. Testing strategy: Unit, integration, browser, compatibility, and manual acceptance tests.
20. Prompt quality evaluation: Dataset, evaluation dimensions, regression tests, and performance targets.
21. Risk register: Major risks, likelihood, impact, and mitigation.
22. Build and deployment: Local development, Chrome testing, packaging, and possible publication.
23. Open questions: Decisions requiring my input.
24. Acceptance checklist: Observable conditions for declaring the MVP complete.

For major architectural decisions, document:

* The decision.
* Alternatives considered.
* Advantages and disadvantages.
* The recommendation.
* The reason for the recommendation.
* Any unresolved assumptions.

Clearly distinguish verified information from assumptions.
Use official documentation for changing facts such as API limits, available models, Chrome permissions, and

platform requirements.
Phase 1 restrictions
During Phase 1:

* Inspect files and configurations as needed.
* Research relevant documentation.
* Create or update `plan.md`.
* Validate the plan for contradictions, missing tasks, unrealistic assumptions, and unnecessary scope.

Do not implement application features.
Do not generate the full application source code.
Do not install dependencies merely to begin implementation.
Do not create unnecessary files.
Do not change existing application configuration without authorization.
Do not make irreversible changes.
Do not silently make major decisions that materially affect security, cost, deployment, or privacy.
After creating `plan.md`, summarize:

1. The proposed architecture.
2. The recommended initial provider and the evidence supporting that choice.
3. The major technical risks.
4. The important decisions requiring my input.
5. The recommended implementation sequence.

Then STOP. Wait for my explicit approval before proceeding to Phase 2.
PHASE 2 — IMPLEMENTATION AFTER APPROVAL
Only after I explicitly approve `plan.md`:

1. Re-read the approved plan.
2. Inspect the repository again.
3. Identify the first implementation milestone.
4. Implement one coherent milestone at a time.
5. Write and run tests for the milestone.
6. Fix relevant failures before marking it complete.
7. Update the task checklist in `plan.md`.
8. Document material deviations and their reasons.
9. Keep the MVP scope controlled.
10. Report actual progress, test results, unresolved issues, and the next milestone.

If a critical architectural assumption is disproven, stop and explain the issue before making a significant architectural change.
Do not interpret approval of the plan as permission to ignore its constraints.
PHASE 3 — VALIDATION AND DELIVERY
Once the implementation is complete:

* Run automated tests.
* Run type checks and linting.
* Build the extension.
* Verify that the build can be loaded into Chrome.
* Provide installation and testing instructions.
* Test prompt capture and replacement.
* Test undo and restoration.
* Test supported editors and document unsupported ones.
* Test Groq and Gemini integrations.
* Test invalid keys, rate limits, exhausted quotas, timeouts, and network failures.
* Verify that provider failure does not lose the original prompt.
* Verify that enhancement never submits the prompt.
* Check for accidentally exposed credentials.
* Review permissions and external data transmission.
* Verify that documentation reflects the actual implementation.
* Report all unresolved limitations honestly.

Never claim a test was executed or passed unless the corresponding test was actually run and its result verified.
10. ACCEPTANCE CRITERIA
The MVP is complete only when:

* The extension can be built and loaded into Chrome.
* A user can enhance text in a supported editor using an intentional action.
* The enhanced prompt replaces the correct input without automatically submitting it.
* The original prompt can be restored.
* Short prompts remain concise when expansion is unnecessary.
* Complex prompts receive relevant improvements.
* The enhancement preserves the user's intent and constraints.
* Framework selection is modular and testable.

* Groq and Gemini are supported through separate adapters behind a shared interface.
* Users can configure their own provider credentials for the chosen prototype architecture.
* Credentials are not hardcoded into distributed source code.
* Errors do not destroy the original prompt.
* The extension does not make unauthorized provider requests or silently switch providers.
* Essential automated tests pass.
* The README explains setup, configuration, permissions, limitations, testing, and provider integration.
* The project contains no placeholder functionality falsely represented as complete.

Add further acceptance tests in `plan.md` wherever necessary.
11. CONSTRAINTS AND PRIORITIES
Apply the following priorities in descending order:

1. Preserve the user's intent and original text.
2. Protect privacy and credentials.
3. Make the core workflow reliable.
4. Improve prompt quality measurably.
5. Support common editable fields.
6. Keep the user experience simple.
7. Maintain modularity and extensibility.
8. Minimize operating costs.
9. Avoid unnecessary complexity.

Additional rules:

* Never submit a prompt automatically.
* Never force every prompt into a formal framework.
* Never equate verbosity with quality.
* Never invent facts or requirements.
* Never discard the original prompt before a successful replacement.
* Never promise universal website compatibility.
* Never assume free API tiers are unlimited.
* Never expose API keys intentionally through the source bundle.
* Never add costly infrastructure without justification.
* Never claim unverified technical capabilities.
* Never start implementation before explicit approval of `plan.md`.

12. FIRST ACTION
Begin Phase 1 now.
Inspect the repository, investigate the important technical uncertainties, and create a detailed, actionable `plan.md` at the project root.
Review the plan for technical feasibility, missing requirements, security problems, and unnecessary scope.
Provide the concise decision summary described above.
Do not write the extension code yet. Do not install dependencies. Do not proceed to implementation. Wait for my explicit approval of `plan.md`.

3.2 Message 2 — Request for prototype screenshots (after the Phase 1 plan)

generate prototype screenshots and save it in the repo first

3.3 Message 3 — Decisions resolving the open questions; approval to implement

Resolve all open questions in §23 of `plan.md` using the following decisions. Prioritize the fastest path to a working, testable MVP. Make reasonable technical decisions independently, document them, and do not ask me to approve routine implementation choices.

1. Groq first: Approved. Implement Groq as the first provider using the best currently available model for prompt rewriting within its eligible free-tier limits. Keep the provider interface modular so Gemini can be added without changing the enhancement engine.
2. Gemini and data privacy: Defer Gemini integration until after the Groq MVP works. Before adding Gemini, verify the current official terms, free-tier data-use policies, and geographic restrictions. Do not assume my location or that I consent to provider data use. Make Gemini an optional, disabled-until-configured provider. Do not make Gemini a blocker for the MVP.
3. Failed output validation: Never replace the original prompt automatically when validation fails. Offer Copy as the safe fallback. Only show Insert anyway if the user explicitly requests it and the result is not unsafe or structurally invalid. Preserve the original prompt in every failure scenario.
4. Missing information: Do not invent facts, requirements, or context. Do not insert arbitrary placeholders. Preserve uncertainty explicitly and, when useful, instruct the target AI to ask concise clarifying questions before proceeding.

Avoid unnecessary clarification instructions for simple prompts.
5. Website targets: Start with standard HTML textareas, text inputs, and accessible `contenteditable` fields. Use these as the primary test targets, then test one popular AI chat interface if it can be accessed reliably. Do not block development on website-specific integrations. Document unsupported editors and cross-origin iframe limitations.
6. Distribution: Design for personal use first, but keep the architecture suitable for possible future Chrome Web Store publication. Do not build a backend, authentication system, analytics, billing, or other infrastructure unless required for the MVP. Clearly document the security limitations of direct client-side API keys and what would need to change before public distribution.
7. Permissions and live tests: Use the minimum necessary permissions. Do not automatically allow blocked domains, bypass security restrictions, or request broader access without justification. Use mocked provider responses and automated tests by default. If live API tests are possible, make them optional and require valid credentials supplied through session environment variables or an appropriate local secret mechanism. Never commit keys or expose them in logs, source code, build artifacts, or error messages. Do not block implementation if credentials are unavailable.

Execution instructions

* Read `plan.md` and resolve every open question in §23 using these decisions.
* Update the architecture, provider strategy, privacy assumptions, testing plan, risk register, and acceptance criteria accordingly.
* Record these decisions in the plan so they are not repeatedly raised.
* Make no unnecessary architectural changes.
* Implement the smallest complete MVP in milestone order.
* Run availa

3.4 Message 4 — API key (redacted) and the requirements for this report

This is the api key - [REDACTED — a Groq API key was supplied here; it is intentionally omitted from the report]

13. FINAL DELIVERABLE: PROJECT REPORT PDF
At the end of the entire project, after implementation, testing, and prototype validation are complete, generate a polished PDF report named `project_report.pdf`.
This report is a mandatory final deliverable. Do not generate the final report prematurely or stop implementation just to create it.
A. Purpose
The report must document the project's development journey, including my original requirements, the decisions made, the AI prompts used during development, your responses and implementation decisions, and visual evidence of the working prototype.
The report should be suitable for project documentation, demonstration, academic submission, and future reference.
B. Required report contents
Organize the report into the following sections:

1. Cover page
* Project title: AI-Powered Prompt Enhancement Chrome Extension.
* Brief project description.
* Report generation date.
* Project version or milestone.
2. Executive summary
* The problem being solved.
* The proposed solution.
* The final features implemented.
* The technologies and AI providers used.
3. Original user requirements
* Include every substantive prompt I provided in our project conversation.
* Preserve the original wording and meaning.
* Include my initial extension concept, the request for a `plan.md`-first workflow, the Groq and Gemini requirements, the implementation decisions, and the request for this report.
* Keep separate messages as separate entries and display them chronologically.
* Do not invent missing messages or represent rewritten versions as verbatim originals.
4. AI response excerpts
* Include relevant snippets from your responses to my prompts.
* Associate each excerpt with the corresponding user prompt.
* Prefer concise excerpts that capture the architecture, framework choices, provider decisions, implementation strategy, and report requirements.
* Clearly distinguish verbatim quotations from summaries.
* Include enough context to explain why each decision was made.
5. Planning and architecture
* Summarize the final `plan.md`.

* Include the final system architecture diagram.
* Explain the prompt enhancement pipeline, framework registry, extension components, provider interface, and editor integration.
* Document the major design decisions and alternatives considered.
6. Prompt engineering methodology
* Explain how CO-STAR, RISEN, RTF, CREATE, APE, and other applicable techniques informed the enhancement engine.
* Explain how the system selects frameworks according to task type.
* Include representative before-and-after prompt examples.
* Use real test examples where available, with sensitive information removed.
* Explain how intent preservation, conciseness, clarity, and output quality are evaluated.
7. Implementation summary
* Describe the actual implemented features.
* List the main technologies, dependencies, and project structure.
* Explain Groq integration and any Gemini integration that was actually completed.
* Document API-key handling, permissions, error handling, undo behavior, and supported editors.
* Clearly identify any planned features that were not implemented.
8. Prototype screenshots and visual evidence
* Include genuine screenshots of the working extension wherever available.
* Prioritize the extension interface, settings, prompt enhancement workflow, original-versus-enhanced prompt, undo behavior, and error states.
* Capture before-and-after examples demonstrating prompt improvement.
* Include the extension running in Chrome when feasible.
* Use readable, consistently sized images with captions and figure numbers.
* Generate additional prototype mockups only when actual screenshots are unavailable, and label every mockup prominently as a conceptual mockup rather than a working prototype.
* Never fabricate screenshots or claim that a mockup demonstrates tested functionality.
* Do not include API keys, credentials, personal information, or private website content in screenshots.
9. Testing and validation
* Summarize automated tests, integration tests, and manual acceptance tests.
* Include actual results, pass/fail status, and any relevant limitations.
* Document provider failures, invalid keys, quota handling, malformed responses, prompt preservation, and editor compatibility where tested.
* Distinguish tests that were executed from tests that were only planned.
10. Challenges and decisions
* Document significant technical challenges and their solutions.
* Explain important trade-offs involving provider selection, free-tier usage, permissions, privacy, cost, and compatibility.
* Record unresolved risks and known limitations.
11. Installation and usage guide
* Explain how to build and load the extension into Chrome.
* Describe how to configure a Groq API key securely for the implemented architecture.
* Explain how to activate enhancement, restore the original prompt, and troubleshoot common errors.
* Include only instructions that match the actual implementation.
12. Conclusion and future roadmap
* Summarize what was achieved.
* List remaining limitations.
* Suggest prioritized future improvements, including Gemini support if it remains incomplete, additional editor integrations, quality evaluation, and possible Web Store publication.

C. Conversation and prompt accuracy
Use the conversation history and available project artifacts as the sources of truth.

* Preserve the user's original prompts accurately.
* Include the substantive user messages relevant to this project, not unrelated conversations.
* Include concise, relevant excerpts from assistant responses rather than duplicating every lengthy response in full.
* If the complete conversation history is unavailable, report that limitation and use the messages and artifacts that are accessible.
* Never fabricate quotations, implementation decisions, test results, or screenshots.
* Keep a clear distinction between requirements, recommendations, actual implementation, and future work.

D. Prototype image capture
When the extension is operational:

1. Build the extension.
2. Load it into Chrome using the documented development workflow.
3. Exercise the core user journey.
4. Capture screenshots of actual functionality.

5. Save the screenshots in a dedicated report-assets directory.
6. Verify that the screenshots are readable and do not expose secrets.
7. Insert them into the PDF with descriptive captions.

If a browser or screenshot capability is unavailable, use only images that can actually be generated or captured in the available environment. Clearly document what could not be demonstrated. Do not claim that live browser testing occurred if it did not.
E. PDF generation requirements
Generate `project_report.pdf` in the project root.

* Use Python with ReportLab's Platypus document-building components.
* Create a professional, consistent layout with a cover page, table of contents, numbered headings, page numbers, readable typography, figure captions, and appropriate spacing.
* Ensure long prompts, code snippets, diagrams, and tables wrap correctly and remain readable.
* Use syntax-appropriate styling for short code snippets.
* Preserve the visual quality of screenshots without stretching them.
* Include the architecture diagram as a rendered image or another PDF-compatible representation.
* Verify that the final PDF opens successfully and contains all required sections.
* Check for clipped text, missing images, blank pages, unreadable figures, and broken page layouts.
* Store supporting screenshots and generation scripts in an organized directory if they are needed to reproduce the report.
* Do not include API keys or other secrets in the PDF or supporting assets.

F. Final verification and delivery
Before declaring the project complete:

* Confirm that the extension build and relevant tests have been run.
* Confirm that `project_report.pdf` exists and opens correctly.
* Verify that original user prompts and assistant response excerpts are represented accurately.
* Verify that prototype images are genuine and correctly labeled.
* Ensure that incomplete features and unexecuted tests are disclosed.
* Provide the exact location of the report and a direct download link when the environment supports one.
* Provide the extension build location and concise instructions for loading it into Chrome.

Final completion criteria: The project is not considered fully delivered until the working extension, available test results, documentation, and verified `project_report.pdf` have all been produced. If a requirement cannot be completed, explain precisely what is missing rather than claiming completion.

4. AI response excerpts

Quotations in shaded boxes are **verbatim** excerpts from the assistant's replies in this conversation (markdown emphasis markers removed). Text outside the boxes is a **summary** written for this report.

4.1 Response to message 1 — Phase 1 discovery and plan.md

Summary: the repository was inspected (empty, no commits), provider documentation was researched, `plan.md` (24 sections) was written and pushed, and work stopped for approval as instructed. Direct access to official documentation sites was blocked, so facts were tagged.

> Repo is empty (no commits). Direct fetches to `console.groq.com`, `ai.google.dev`, and `developer.chrome.com` are blocked by this session's network policy, so I'm relying on search-engine snapshots of those official pages and marking every changing fact as unverified.
>
> One AI call per enhancement. Classifying the task, choosing techniques and rewriting all happen in one call that returns structured JSON. Checks on the input and the output run locally, with no AI involved.
>
> Privacy, the deciding reason. On Gemini's free tier, Google may use your prompts to improve its products, and human reviewers may read them. Groq keeps logs for at most 30 days, and you can turn on zero data retention in your Groq console.
>
> Against it: Gemini may produce more reliable structured output. The evaluation milestone decides which provider and model you use day to day; "first" doesn't mean "best".
>
> I'm waiting for your approval before writing any code.

4.2 Response to message 2 — prototype screenshots

Summary: static HTML mockups of 11 UI states were rendered to PNG (`docs/prototype/`). Reviewing them exposed a design flaw, which changed the plan.

> Two problems: the caption covers the text box, and the toast hides the end of the prompt. Hiding the user's text is a real UX flaw, not just a mockup issue, so I'll anchor the toast just above the field, and the plan will be updated to match.
>
> They're static mockups for reviewing the plan, not extension code. The host site is an unbranded stand-in chat page.

4.3 Response to message 3 — decisions and implementation

Summary: decisions D1–D7 were recorded in `plan.md`, then milestones M0–M4 and the offline part of M6 were implemented and tested. Two defects were found by tests and fixed (Section 10).

> The message cut off at "Run availa…". I'll assume it meant "run the available tests and report results", which matches the Phase 2 workflow.
>
> I have no Groq key, and the proxy blocks `api.groq.com` (403). Live provider tests will be optional and skipped here, and everything else runs against mocks.
>
> This is a real finding. ProseMirror handles the synthetic paste, but it appended the text instead of replacing it. ProseMirror syncs the selection from a `selectionchange` event that hasn't fired yet when the paste arrives. The E2E test only passed because the fallback chain recovered. I'll fix the cause and make tests assert which strategy succeeded, so a silent fallback can't hide a failure again.
>
> A genuine gap: Spanish→English translation isn't caught, because both use the Latin script. I'll add lightweight stopword-based language detection for common Latin-script languages to the validator.

4.4 Response to message 4 — API key and report

Summary: the key was treated as a secret, never written to disk or used (no request could reach Groq), and this report was produced last, after implementation and testing.

> Noted, and a security point first: that key is now stored in this conversation's transcript. I won't write it to any file, commit, log, build, or report, and I'll redact it wherever your messages are quoted. Rotate it in the Groq console once the project is done.
>
> Live Groq test: the proxy rejected the connection to `api.groq.com` before any request was sent, so sending your key would accomplish nothing, and I won't try to get around the network policy (D7).

5. Planning and architecture

5.1 Summary of plan.md

`plan.md` (24 required sections plus a decision log and implementation notes) defines scope, architecture, provider research, security model, test strategy and acceptance checklist. Facts are tagged [V] verified, [S] search-snapshot of official docs, or [A] assumption. The user's decisions are recorded as binding entries D1–D7:

ID	Decision (user, message 3)	Effect
D1	Groq first, best free-tier model, modular interface	Groq adapter; default `openai/gpt-oss-120b`
D2	Gemini deferred; verify terms first; disabled until configured	Gemini shown as 'not available yet'; milestone M5
D3	Never auto-replace on validation failure; Copy fallback; Insert anyway only if requested and safe	Blocking vs overridable failure classes; opt-in setting, off by default
D4	No invented facts or placeholders; clarifying questions when useful	System rules 4–5; placeholder checks in eval
D5	Start with textarea/input/contenteditable; real chat UI best-effort	Fixture-based E2E; real sites documented as untested
D6	Personal use; Web Store-ready; no backend	BYOK direct calls; publication checklist in README
D7	Minimum permissions; no bypassing restrictions; mocks by default; optional live tests via env	Mock provider; `npm run eval:live` skipped without key

5.2 System architecture

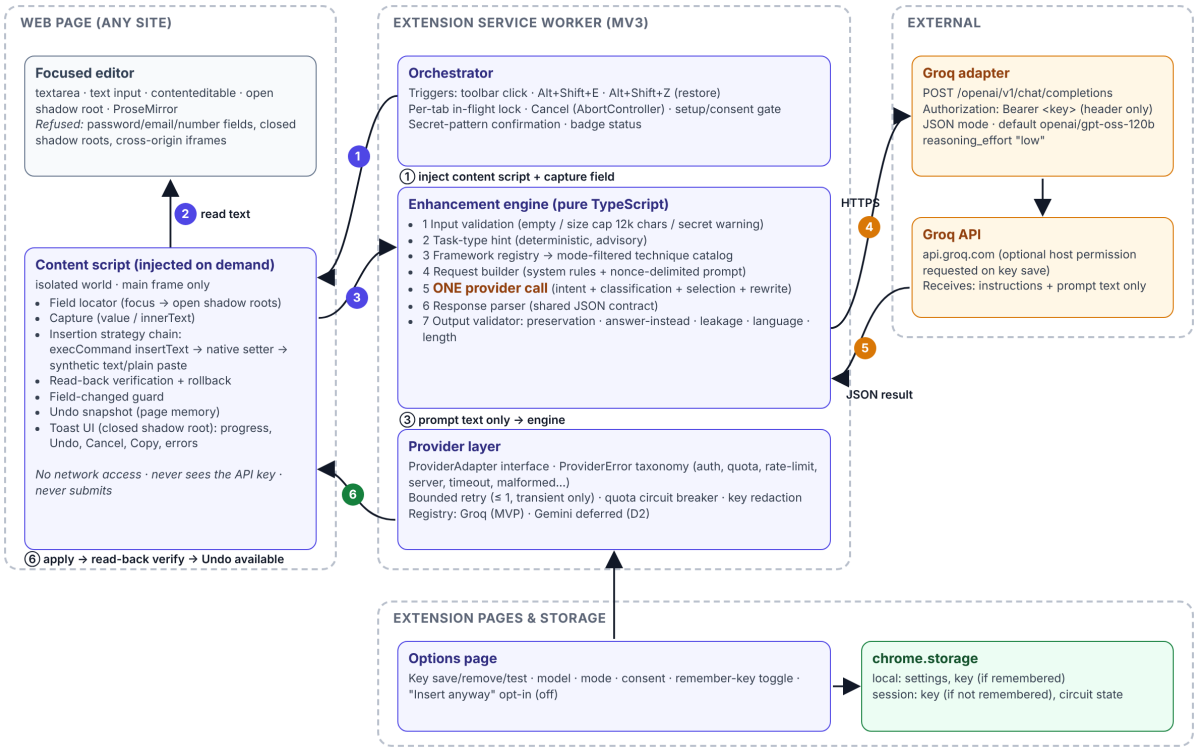


Figure 1. System architecture and data flow. ① user gesture → service worker injects the content script and captures the focused field; ② read text; ③ only the prompt text reaches the engine; ④ one HTTPS request to Groq; ⑤ JSON result parsed and validated; ⑥ content script inserts, verifies by reading back, and keeps Undo.

5.3 Components

Component	Responsibility	Explicitly not responsible for
Service worker	Triggers, in-flight lock, cancel, setup/consent gate, engine + provider calls, retry, circuit breaker, badge	DOM access
Content script (on demand)	Locate field, capture, insert via strategy chain, read-back verify, rollback, undo, toast	Network calls, API keys, submitting
Engine (pure TS)	Validation, hint, catalog selection, request building, parsing, output validation	Chrome APIs, fetch (injected)
Provider adapters	Wire format, model list, error mapping to shared taxonomy	Prompt content decisions
Options page	Key, model, mode, consent, test connection, permission request	Running enhancements

5.4 Enhancement pipeline

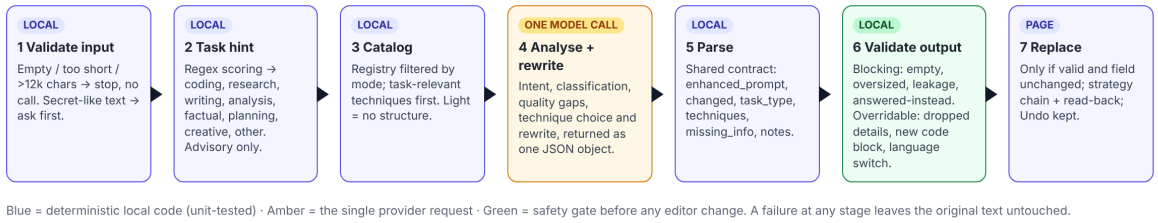


Figure 2. Enhancement pipeline: six local, deterministic stages around a single provider call.

Intent analysis, classification, quality analysis, framework choice and rewriting are combined into **one** model call to halve latency and quota use and to limit the number of times the user's text passes through a model. Each stage remains a separate field in the JSON response, so it can be inspected and tested. The response contract is identical for all providers:

```
{"enhanced_prompt": "…", "changed": true, "task_type": "coding",
 "techniques": ["explicit_instructions", "output_format"],
 "missing_info": ["which columns contain dates"], "notes": "…"}
```

5.5 Provider interface

```
interface ProviderAdapter {
  readonly id: string;
  readonly displayName: string;
  readonly hostPermission: string;   // requested only when a key is saved
  readonly models: readonly ModelInfo[];
  readonly defaultModel: string;
  // Returns raw model text; throws ProviderError only.
  complete(request: ProviderRequest, ctx: ProviderCallContext): Promise<string>;
  listModels(ctx: { apiKey: string; signal: AbortSignal }): Promise<string[]>;
}
```

A second (mock) adapter in the unit tests runs through the same pipeline, demonstrating that the engine has no Groq-specific code. Adding Gemini means a new folder under `src/providers/`, a registry entry, an optional host permission and consent text.

5.6 Major design decisions and alternatives

Decision	Alternatives considered	Reason
On-demand injection (activeTab + scripting)	Static content scripts on all sites; <all_urls>	No code runs on any page until the user acts; no broad host access
No popup; toolbar click enhances directly	Popup with mode picker	A popup intercepts the click and adds a step to the core flow

Decision	Alternatives considered	Reason
Network calls in the service worker	Fetch from content script	Avoids page CSP/CORS and keeps keys out of the page-adjacent world
One model call	Separate classify + rewrite calls; LLM judge	Latency, quota, fewer passes over private text; judges are not guarantees
Vanilla TS + esbuild, zero runtime deps	React/Preact, Vite+CRXJS, provider SDKs	Tiny UI surface; no framework conflicts in pages; full control of requests
BYOK direct calls	Backend proxy with hosted key	No infrastructure or cost for personal use; proxy documented for public release
Verified insertion with rollback	Write value and hope	Editors silently reject changes; read-back proves success or restores the original

6. Prompt engineering methodology

6.1 Frameworks as checklists, not templates

Each framework and technique is a registry entry with intended use, task types it fits, when to avoid it, allowed modes, a one-line instruction, limitations and provenance. The model receives a compact catalog and is told: "Frameworks below are checklists for your thinking, not templates. Do not add framework labels such as 'Context:' or 'Objective:' unless headings genuinely help a long, multi-part task." Acronym expansions are documented as conventions, with known conflicts recorded.

Framework	Expansion used	Selected for	Modes	Provenance / caveat
CO-STAR	Context, Objective, Style, Tone, Audience, Response	writing, creative	Balanced, Deep	Popularised 2023 (GovTech Singapore write-up); informal
RISEN	Role, Instructions, Steps, End goal, Narrowing	planning, analysis, coding	Deep	'N' also expanded as 'Novelty' elsewhere
RTF	Role, Task, Format	short tasks needing a format	Balanced, Deep	Community convention
CREATE	Context, Role, Explicit, Action, Target audience, Exclusions	content creation	Deep	Conflicting expansions exist; user's list used
APE	Action, Purpose, Expectation	short goal-oriented requests	all	Name collides with 'Automatic Prompt Engineer' (2022)

Twelve further techniques: ambiguity reduction, explicit instructions, output format, success criteria, scope boundaries, role/perspective, step sequencing, task decomposition, context surfacing (ask, never invent), few-shot (only user-supplied examples), source-material delimitation, and untrusted-input handling.

6.2 How selection depends on task type and mode

- A deterministic, regex-based **task hint** (coding, research, writing, analysis, factual, planning, creative, other) orders the catalog; it is advisory and the model may override it.
- The **mode filters** what may be offered: Light offers only ambiguity reduction, explicit instructions, APE and untrusted-input handling (no structure); Balanced adds format, constraints, success criteria and light frameworks; Deep adds decomposition, RISEN, CREATE and few-shot.
- The model returns the IDs it applied; unknown IDs are discarded by the parser. Unit tests verify, e.g., that Light never offers CO-STAR or decomposition and that writing tasks rank CO-STAR above success criteria.

6.3 Before-and-after examples

Important: no real Groq output exists for this report (the API was unreachable). The examples below are labelled by origin: (a) is the canned response of the local mock server used in the screenshots; (b)–(e) are hand-written synthetic candidates from the offline regression suite, showing what the validator accepts and rejects. None is presented as model output.

Example	Before (user prompt)	After / candidate	Outcome
(a) mock demo — coding	write a python script that cleans up my csv file, remove duplicates and fix the dates, it has like 50k rows	Write a Python script that cleans a CSV file with about 50k rows. Requirements: 1. Remove duplicate rows. 2. Fix the dates … Before writing the code, ask me concise clarifying questions about anything critical you can't infer … Return the complete script …	Inserted (Figure 'Prompt enhanced')

Example	Before (user prompt)	After / candidate	Outcome
(b) synthetic — factual, good	What is the boiling point of water at sea level in Fahrenheit?	(unchanged)	Accepted: 'Already clear — no changes'
(c) synthetic — factual, bad	What is the boiling point of water at sea level in Fahrenheit?	Context: You are an expert physicist. Objective: ... Style: academic. Tone: formal. Audience: students. Response: a detailed paragraph with citations ...	Rejected: length ratio and inappropriate technique (co_star)
(d) synthetic — insufficient info, bad	fix the bug	Fix the bug in [insert your code here] which throws [describe error].	Rejected: placeholder (D4)
(e) synthetic — language, bad	Escribe un correo breve a mi equipo para recordarles la reunión del viernes a las 10:00.	Write a short email to my team reminding them about Friday's meeting at 10:00.	Rejected: language switch (overridable)

6.4 How quality is evaluated

- **Runtime gate (every request):** output validator — preservation of numbers, quotes, code, URLs, emails, paths, @mentions; answer-instead-of-rewrite detection; leaked delimiters/instructions; language/script switch; oversize; length-ratio warning per mode.
- **Dataset:** `eval/dataset.json`, 30 synthetic prompts across 17 categories (factual, underspecified coding, research, writing, creative, complex analysis, explicit constraints, already-good, insufficient info, adversarial quoted text, non-English, token preservation, planning, modes).
- **Deterministic checks per item:** mustContain / mustNotContain / placeholder regexes, max length ratio (conciseness), acceptable task types, techniques that should not be used, same language.
- **Human rubric** (`eval/rubric.md`): intent preservation, clarity, specificity, relevance of structure, output-format clarity (1–5) and invented requirements (yes/no). Thresholds: intent ≥ 4.5, clarity ≥ 4, no invented requirements on ≥ 90% of items. Scores describe one dataset and model version — not a guarantee.
- **Status:** the offline regression (14 good/bad synthetic cases) runs in CI and passes. The live run and human scoring have **not been executed** (no API access).

7. Implementation summary

7.1 Implemented features

- **Triggers:** toolbar click, `Alt+Shift+E` (enhance), `Alt+Shift+Z` (restore); rebindable at chrome://extensions/shortcuts.
- **Capture:** focused element, followed into open shadow roots; refuses password/email/tel/number/credit-card, read-only and disabled fields; main frame only.
- **Insertion:** strategy chain — `execCommand('insertText')` (fires real input events, keeps native undo), prototype value setter + input event (React-safe), synthetic `text/plain` paste (ProseMirror/Lexical first). Every attempt is verified by reading the field back; failures restore the original and offer Copy.
- **Protection:** per-tab in-flight lock; Cancel/Esc via AbortController; 20 s timeout; field-changed guard (asks before overwriting new typing); secret-pattern confirmation; consent gate.
- **Undo:** toast Undo (10 s), restore command any time on the page, native Ctrl+Z in textareas; asks before restoring over later edits.
- **Errors:** auth, forbidden, missing permission, quota exhausted (circuit breaker until reset, 'Try anyway' override), rate limited, invalid request, model unavailable, server, timeout, network, cancelled, malformed response — each with a plain message that confirms the text was not changed.
- **Settings:** Groq key (masked, save/remove, test connection that checks the model allowlist against `/models`), model, mode, remember-key toggle, Insert-anyway opt-in, consent with versioned disclosure.

7.2 Technologies, dependencies and structure

Area	Choice
Language / platform	TypeScript 6 (strict), Chrome Manifest V3, minimum Chrome 116
Build	esbuild via `scripts/build.mjs` (production `dist/`, test `dist-test/`)
Tests	Vitest 5 (unit), Playwright 1.63 (browser, real unpacked extension), ESLint 10 + typescript-eslint
Runtime dependencies	None — the shipped bundle contains only project code
Test-only dependencies	React 19 and ProseMirror (editor fixtures only)

```
src/background/   service worker: index, orchestrator, inject, badge
src/content/      field-locator, insertion, toast, clipboard, index
src/engine/       pipeline, input-validation, classify-hint, request-builder,
                  response-parser, output-validation, instructions/, frameworks/
src/providers/    types, errors, retry, circuit, registry, groq/{adapter,config}
src/settings/     schema, store, credentials
src/options/      options.html/.ts/.css
tests/unit/ tests/e2e/ tests/fixtures/   eval/   scripts/   report/
```

7.3 Groq integration (and Gemini status)

- `POST https://api.groq.com/openai/v1/chat/completions`, key only in the `Authorization: Bearer` header, `response_format: json_object`, temperature 0.2, `max_completion_tokens` scaled to input (512–4096) plus 1024 reasoning headroom for reasoning models.
- Models (allowlist, switchable): `openai/gpt-oss-120b` (default, `reasoning_effort: low`), `llama-3.3-70b-versatile`, `llama-3.1-8b-instant`. **Model IDs and limits are unverified against live docs** and are checked at runtime via `/models`.
- 429 responses are classified as daily quota vs transient rate limit from the message text; `retry-after` / `x-ratelimit-reset-*` headers are parsed.
- **Gemini: not implemented** (decision D2). It appears in settings as 'not available yet'.

7.4 API keys, permissions and data

- Permissions: `activeTab`, `scripting`, `storage`; optional host `https://api.groq.com/*` requested when the key is saved. Enforced by `scripts/check-dist.mjs` and a production-load test.
- Keys stored under a separate storage key, never in settings, never in URLs/logs; errors redacted; content script never receives the key. Session-only storage option.
- Sent to Groq: instructions, mode, task hint, and the field's text inside per-request nonce delimiters. Never sent: page URL/title, page content, history, identifiers.
- Documented limitation: chrome.storage is not encrypted at rest; anyone with access to the profile can read a saved key.

7.5 Planned but not implemented

Item	Status / reason
Gemini adapter, provider switching, 'Try with other provider'	Deferred by D2 until terms/data-use/region rules are re-verified
Live Groq evaluation, latency measurement, human scoring	Harness ready (`npm run eval:live`); API unreachable from build environment
Real-site testing (ChatGPT, Claude.ai, Gemini web, Perplexity, GitHub)	Needs a logged-in browser; not available
Lexical, Slate, Quill, CodeMirror fixtures	Deferred (D5 scope); covered only by the generic verified strategy chain
Diff view, history, presets UI, custom frameworks, selection-only enhancement	Post-MVP by design

8. Prototype screenshots and visual evidence

Provenance. Figures in 8.1–8.3 are **genuine screenshots** captured automatically by Playwright from the actual extension build loaded into Chromium (`npm run screenshots`). The demo page is a local, unbranded test fixture. Because `api.groq.com` was unreachable, the extension was pointed at a **local mock Groq-compatible server** (test build only); the enhanced text shown is that server's canned response, not real model output. Headless capture shows page content only, so the browser toolbar and badge are not visible. The only API key ever shown is a fake test value (`test-key-not-a-real-credential`, masked).

8.1 Core journey: before, enhancing, after, undo

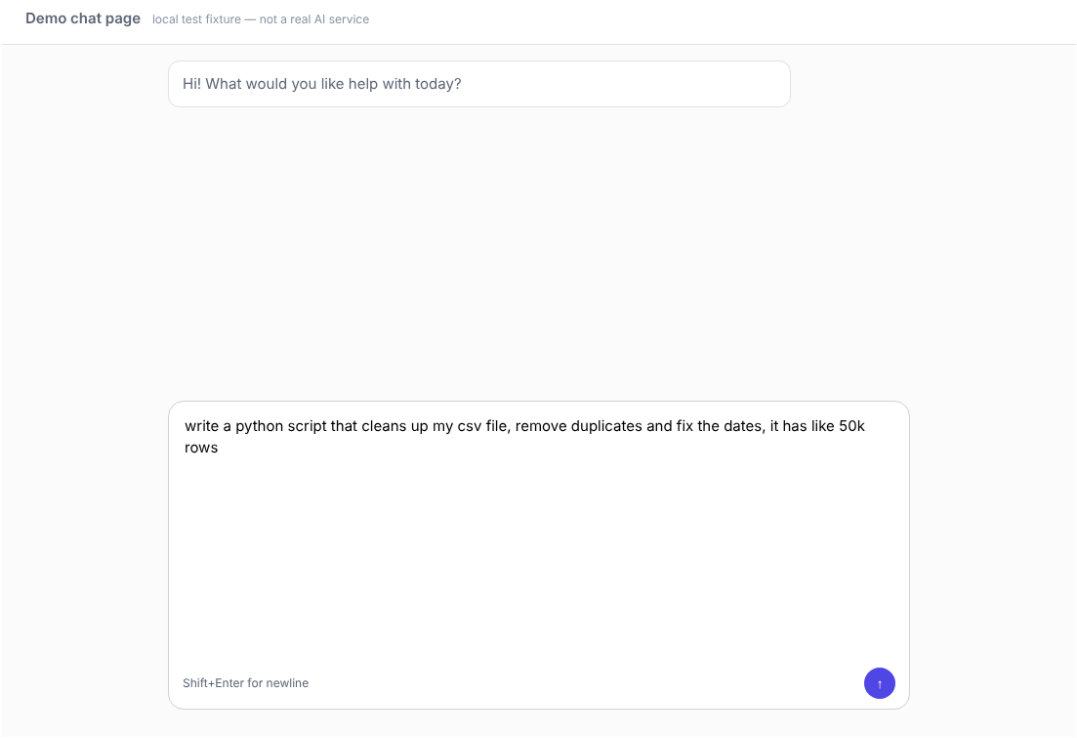


Figure 3. Before: a rough prompt typed into a chat-style text box (local demo fixture).

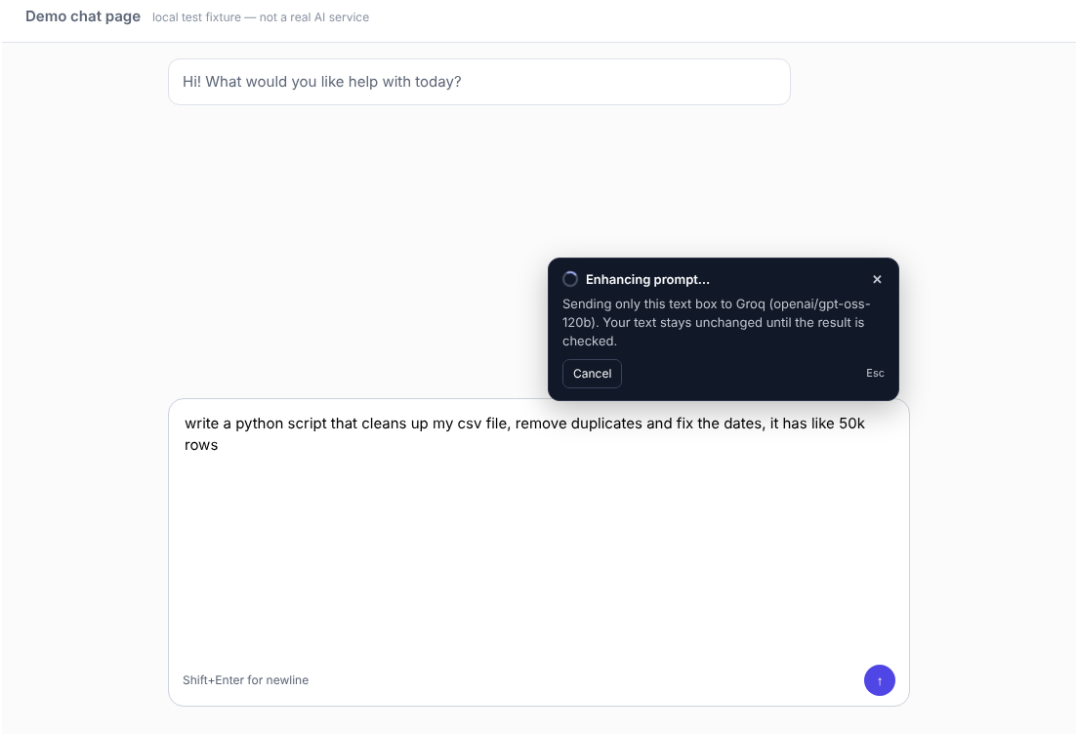


Figure 4. Enhancing: progress toast anchored above the field with Cancel/Esc; the text is unchanged until the result is validated.

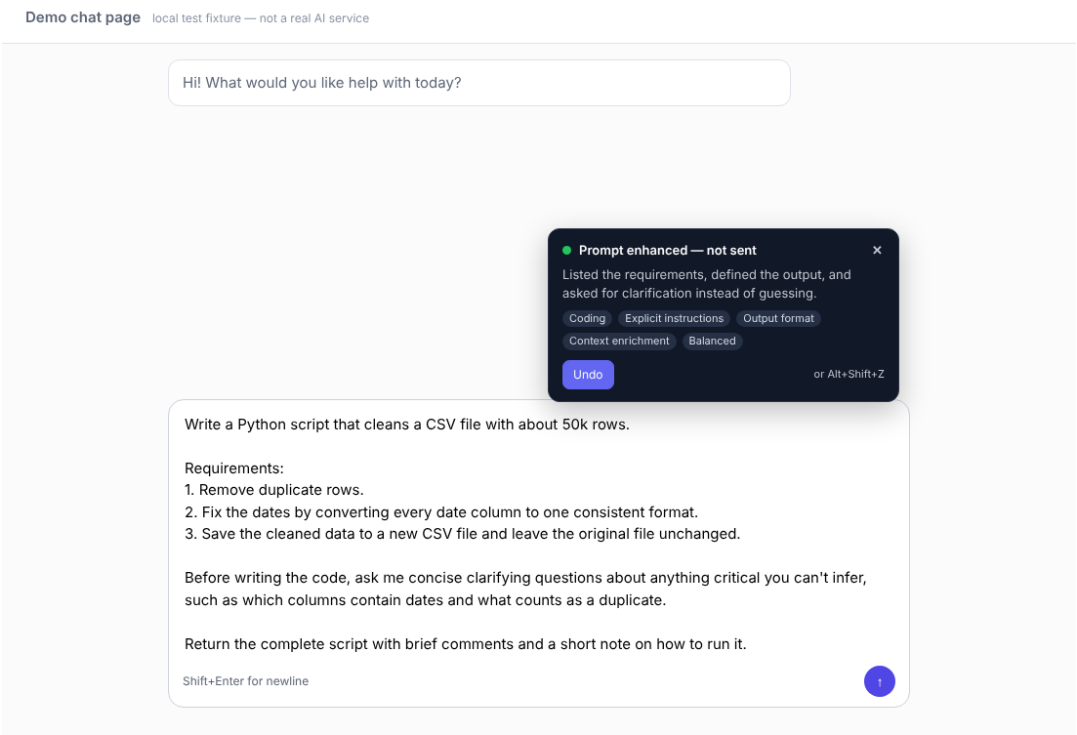


Figure 5. After: the enhanced prompt (mock response) replaces the text in the same field; 'not sent'; task/technique chips; Undo.

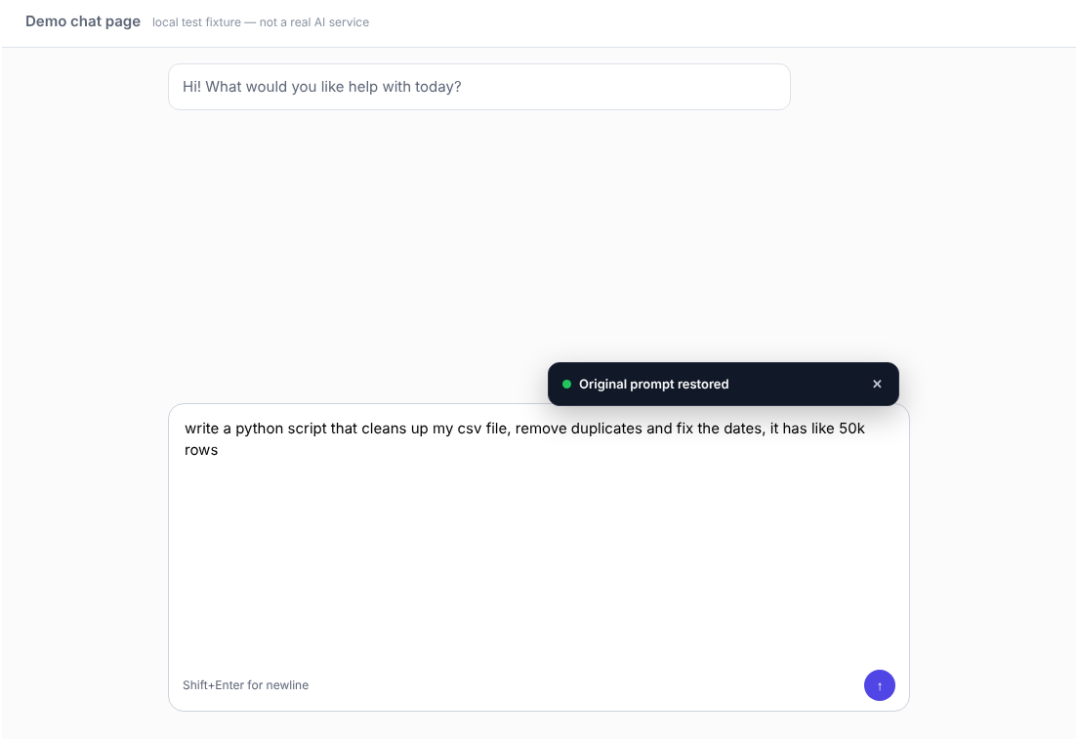


Figure 6. Undo: the original prompt is restored exactly.

8.2 Safety and error states

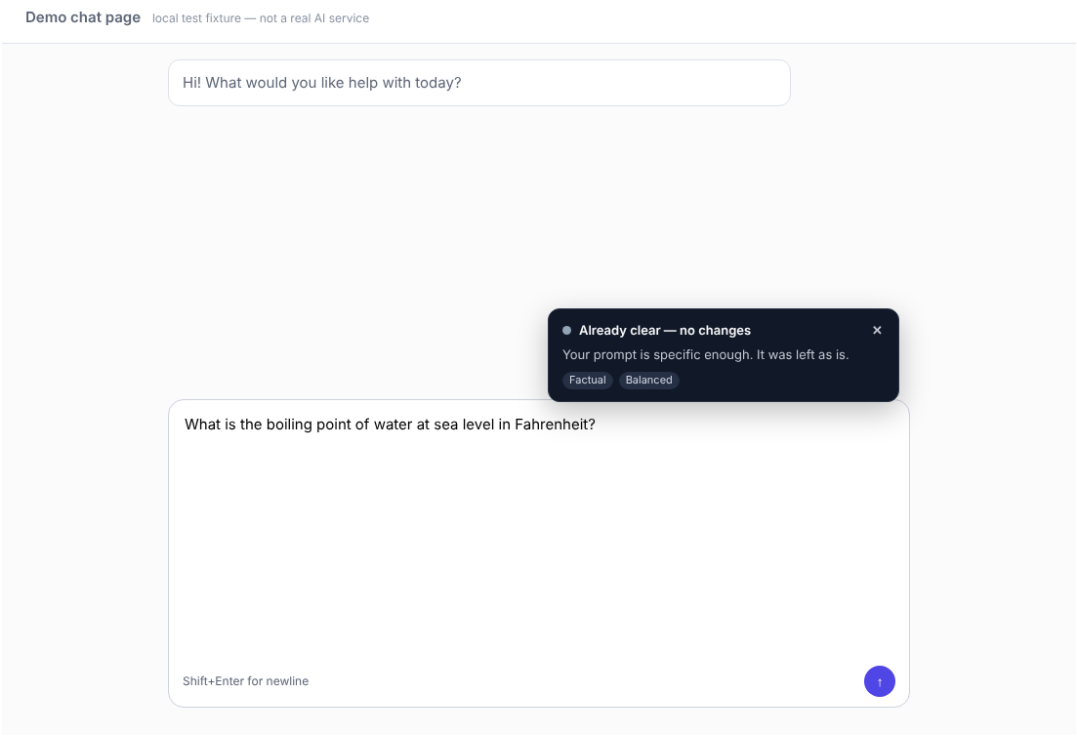


Figure 7. Already-clear prompt: left unchanged ('effectiveness, not length').

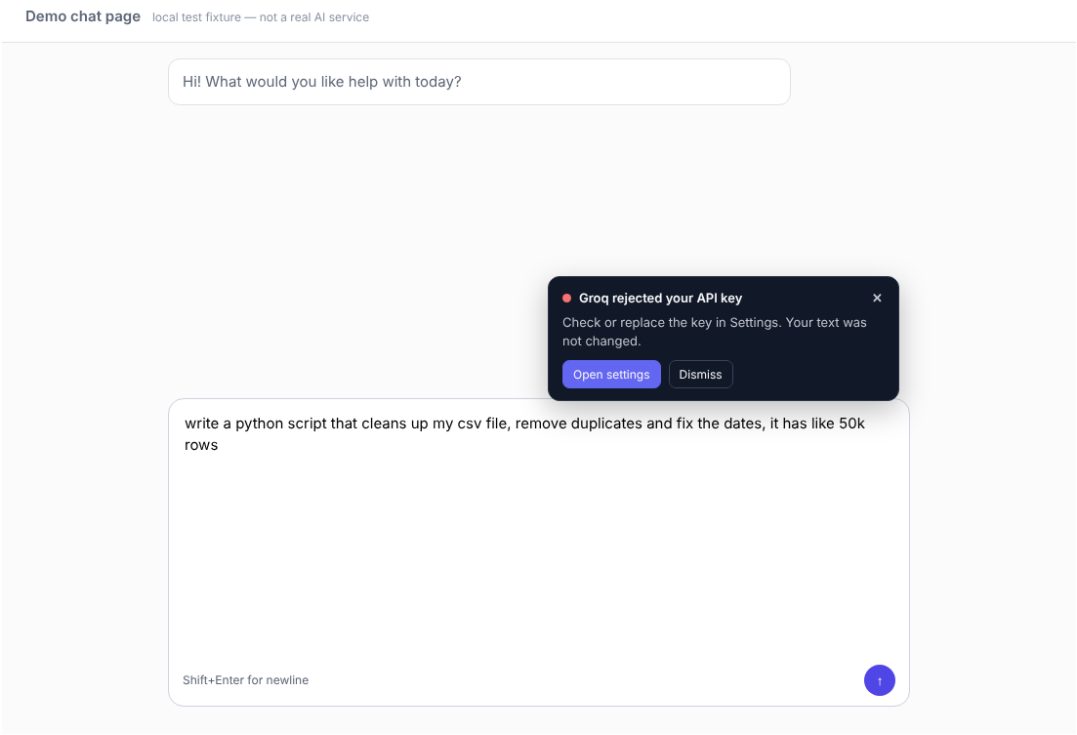


Figure 8. Invalid API key (mock 401): clear message, Open settings, original untouched.

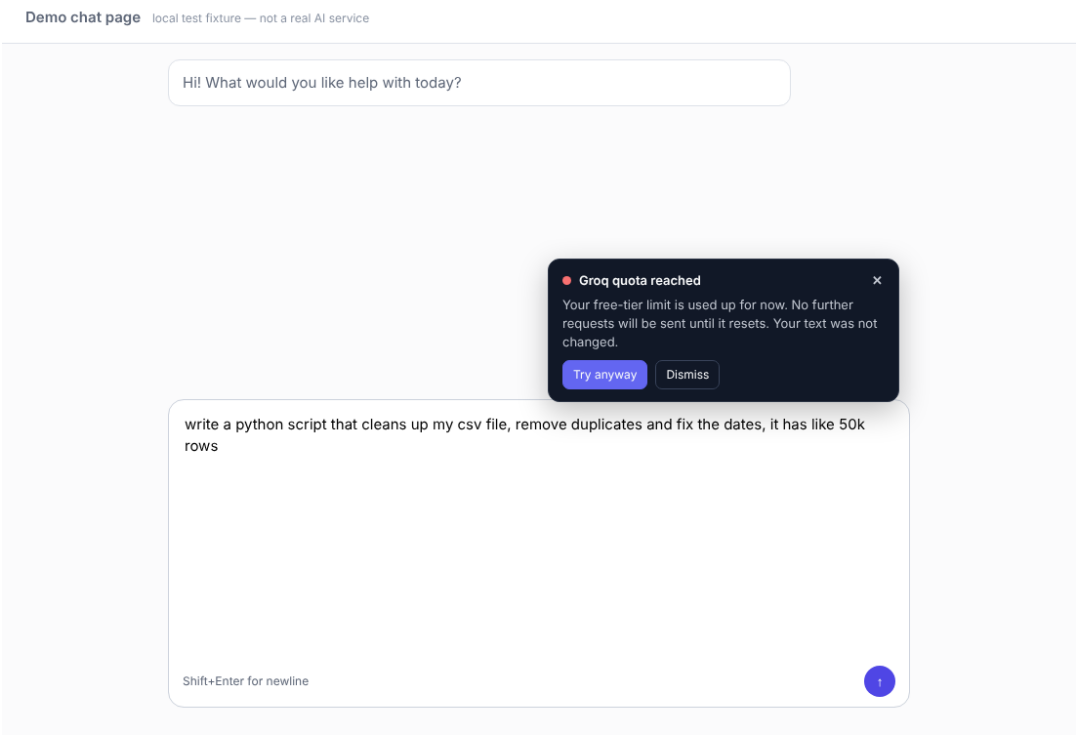


Figure 9. Daily quota reached (mock 429 RPD): circuit breaker engaged; explicit 'Try anyway' override.

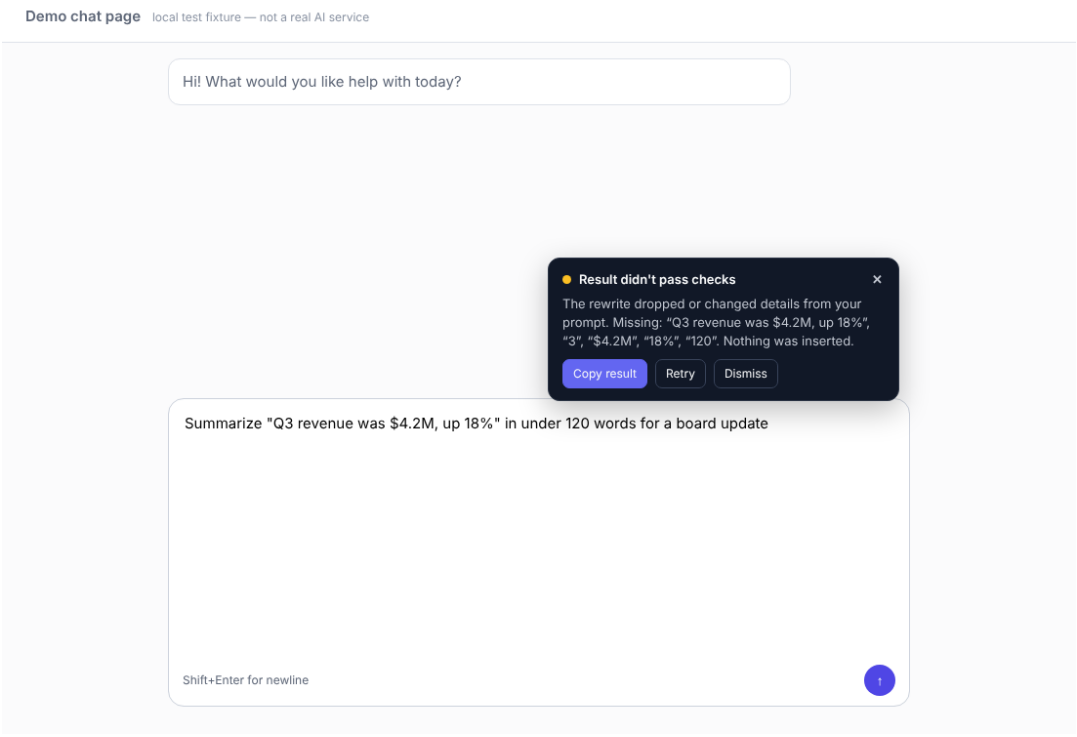


Figure 10. Validation failure: the rewrite dropped quoted text and numbers, so nothing is inserted; Copy offered (Insert anyway is off by default).

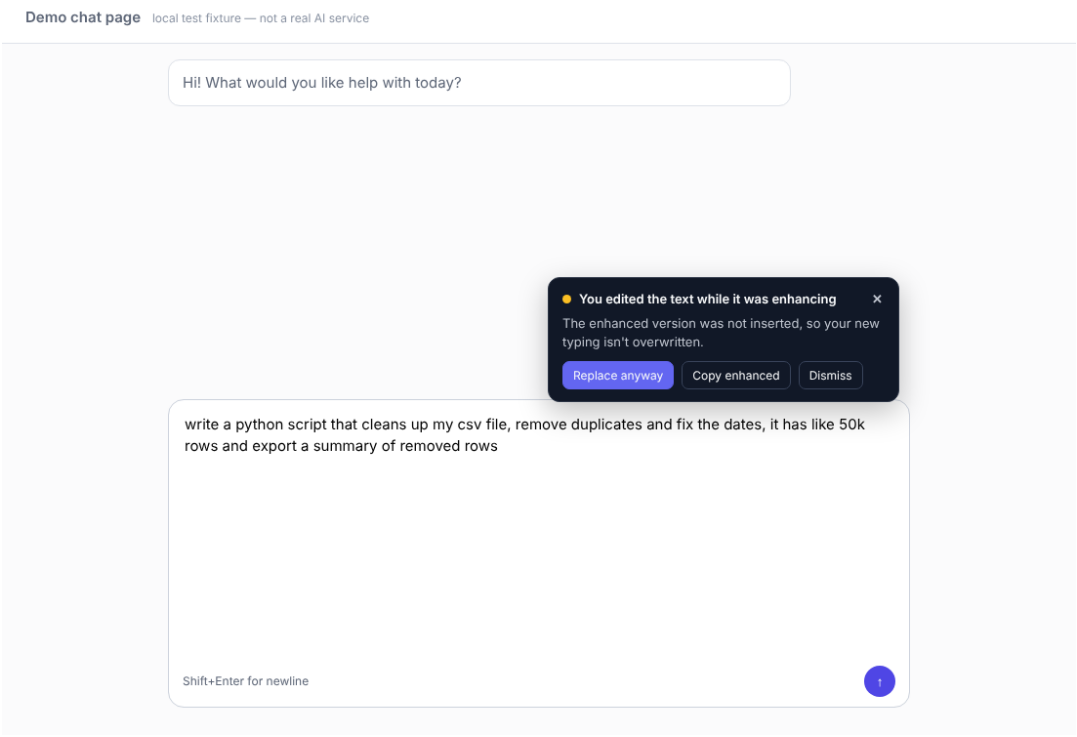


Figure 11. User typed during the request: the enhancement is not inserted; Replace anyway / Copy enhanced.

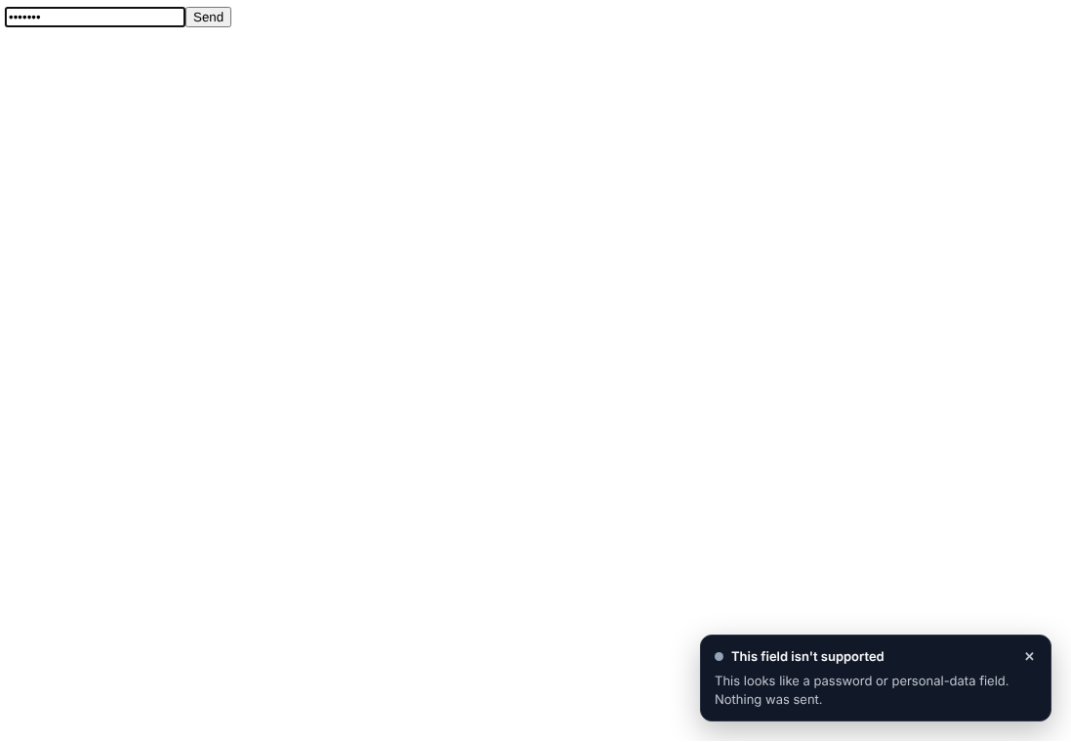


Figure 12. Password field: refused; nothing is sent.

8.3 Settings and installation in Chromium

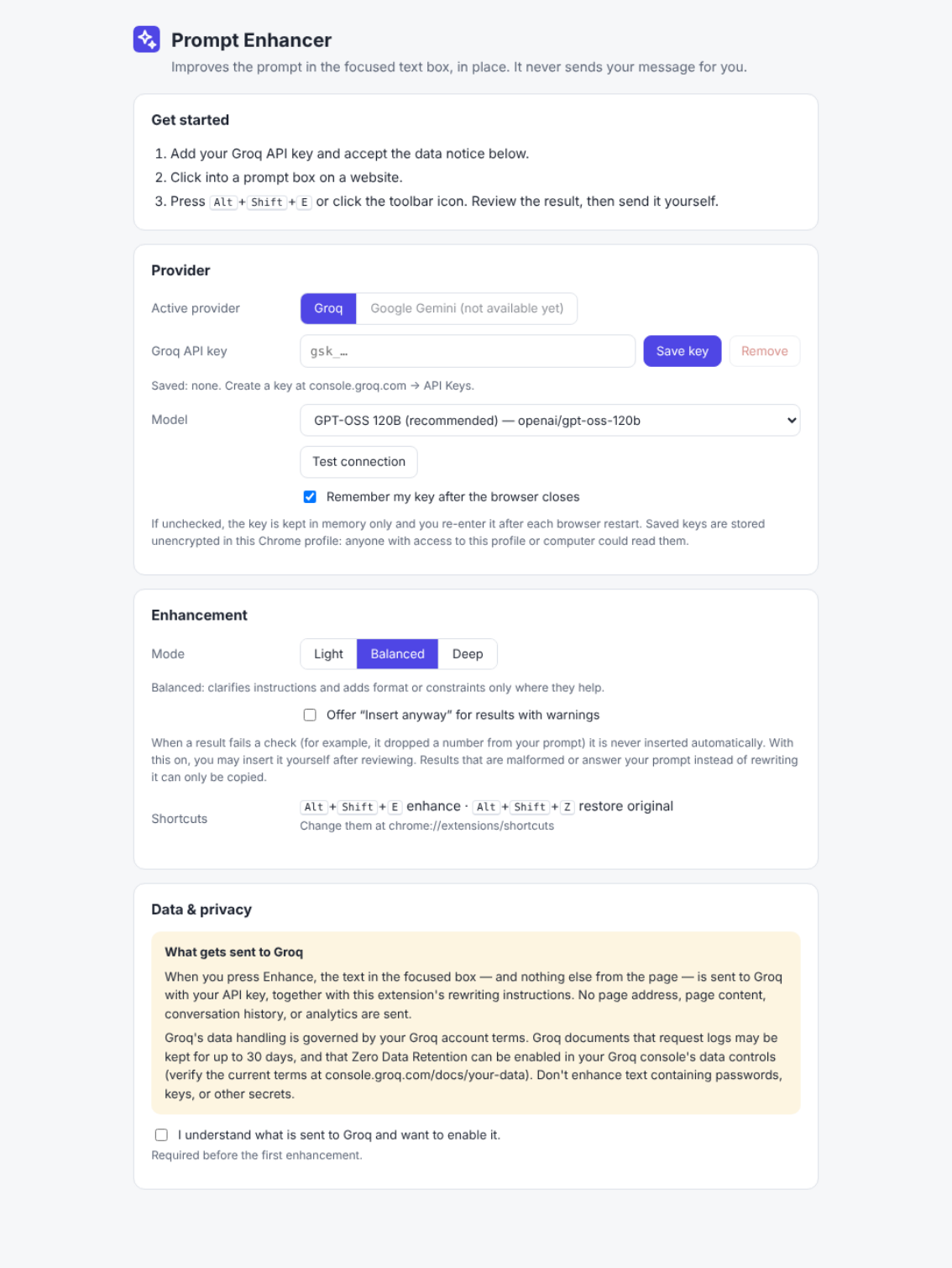


Figure 13. Options page on first run: get-started steps, data disclosure, consent required.

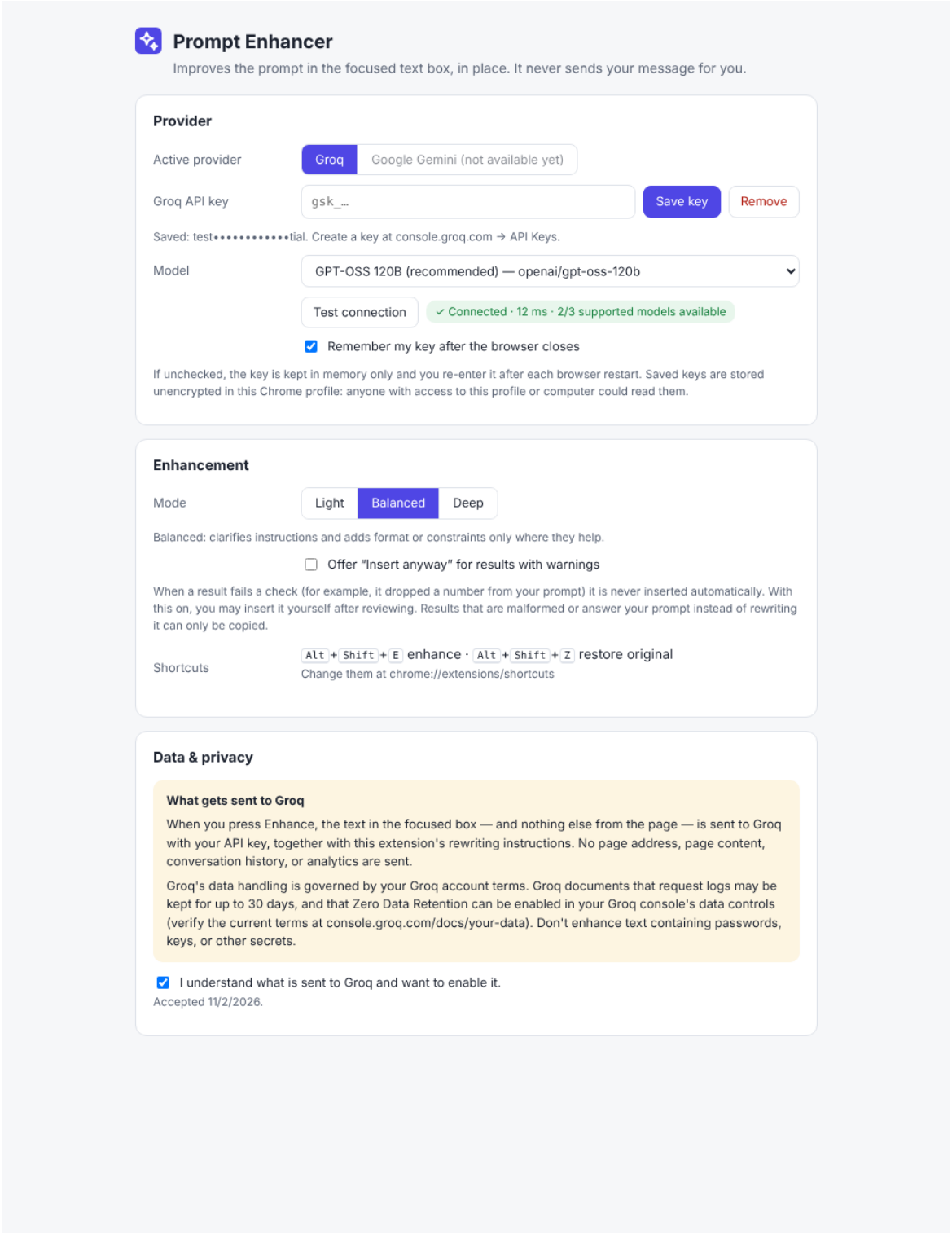


Figure 14. Options page configured: masked key, model, connection test (mock), mode, Insert-anyway opt-in, consent.

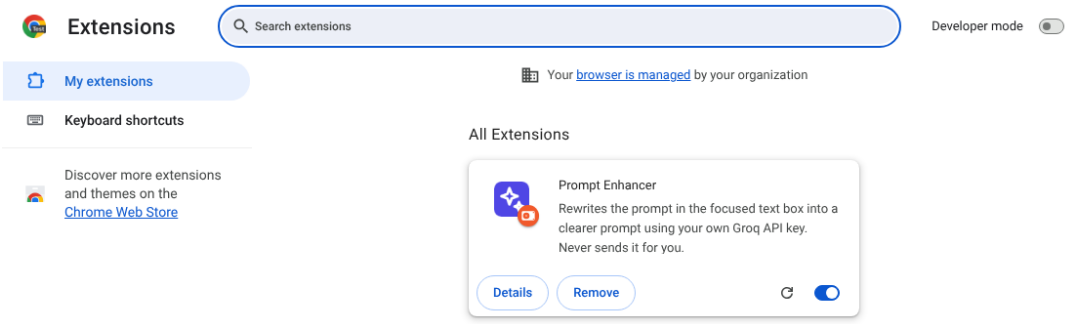


Figure 15. Production build (`dist/`) loaded as an unpacked extension in Chromium (chrome://extensions).

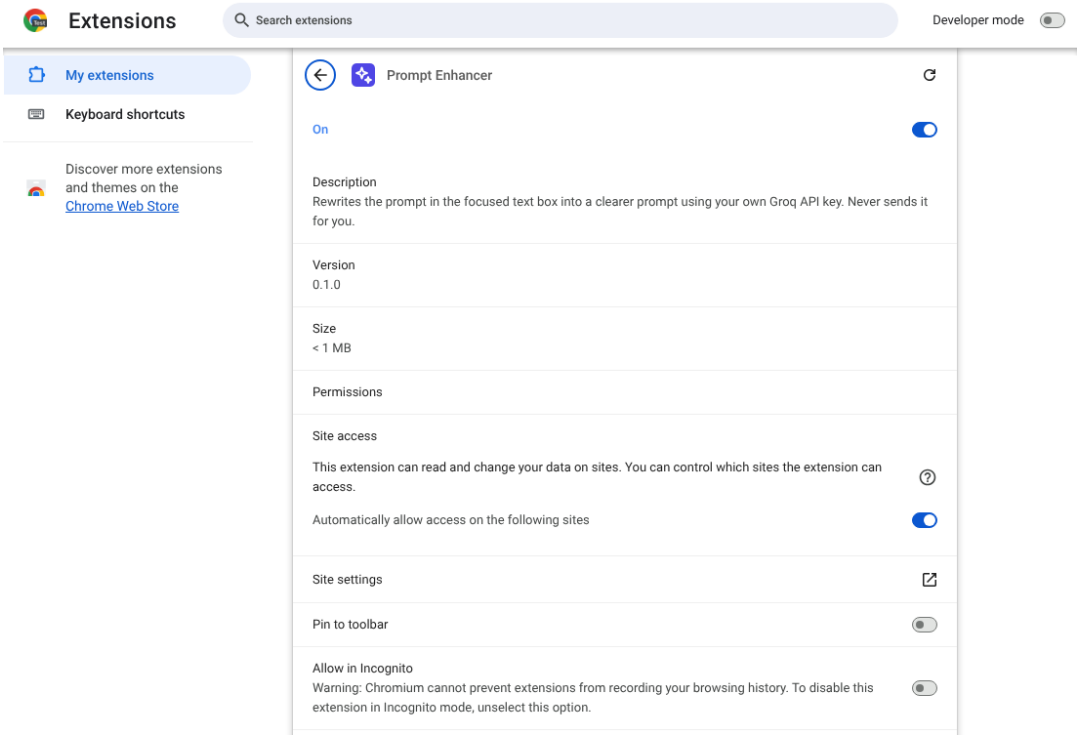


Figure 16. Extension details page for the production build in Chromium.

8.4 Conceptual mockups from the planning phase

These images were produced in response to message 2, **before any extension code existed**. They are design prototypes only and do not demonstrate tested functionality. Comparing the first mockup below with the genuine 'After' screenshot in Section 8.1 shows the design change they prompted: the toast now sits above the field instead of covering it.

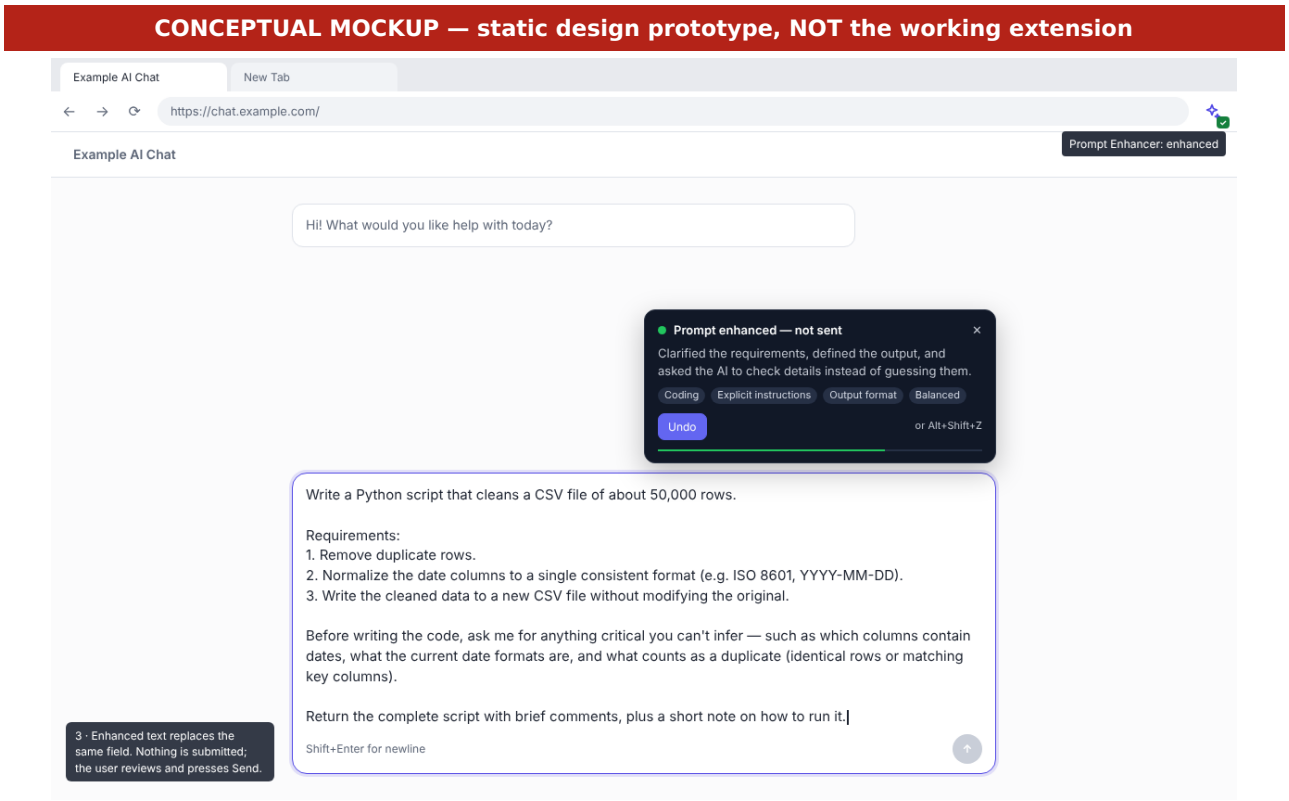


Figure 17. Conceptual mockup: planned 'enhanced' state (static HTML).

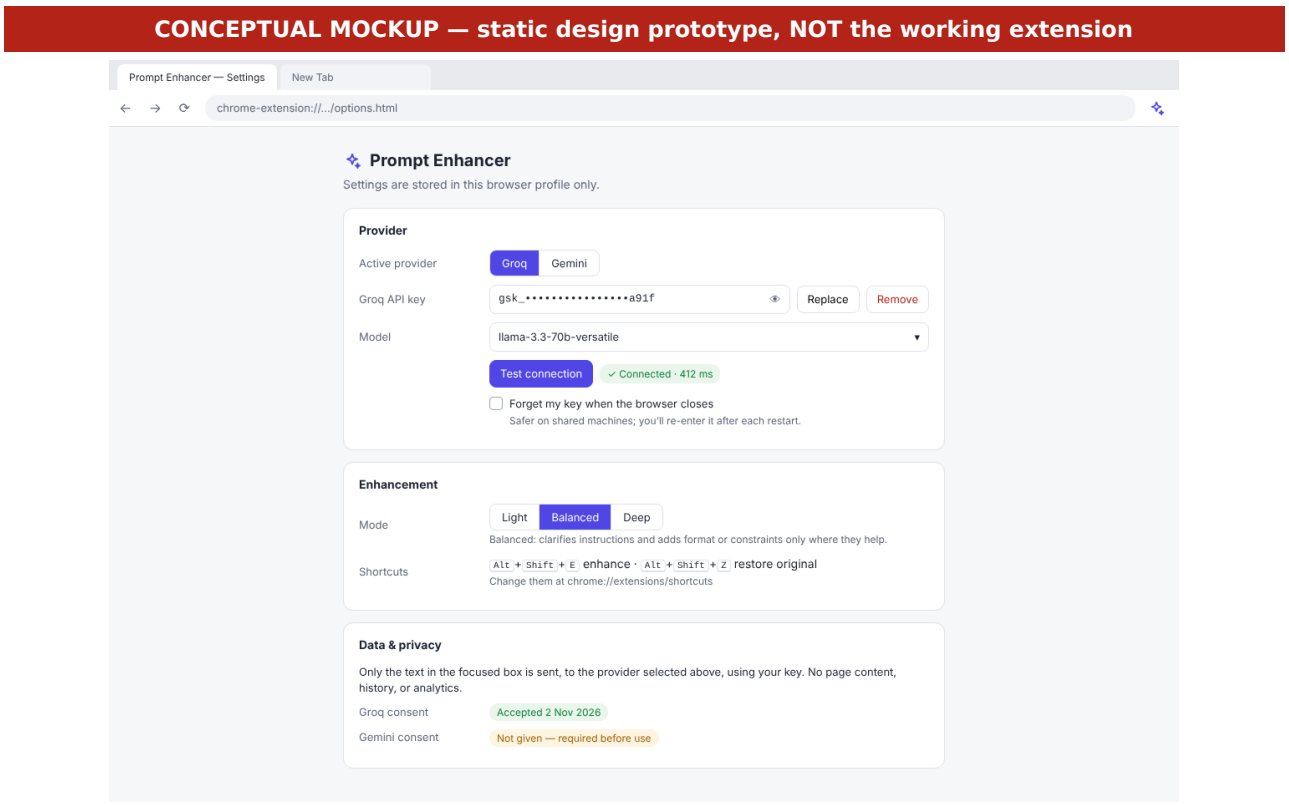


Figure 18. Conceptual mockup: planned settings page (static HTML).

9. Testing and validation

9.1 Executed — results

Check	Command	Result (executed 2026-11-02)
Type checking	`npm run typecheck`	PASS — 0 errors
Lint	`npm run lint`	PASS — 0 problems
Unit tests (Vitest)	`npm test`	PASS — 113 / 113 in 8 files
Browser tests (Playwright, real unpacked extension in Chromium)	`npm run test:e2e`	PASS — 36 / 36 (3 screenshot-only specs skipped by design)
Production build + dist safety check	`npm run build` + `check-dist.mjs`	PASS — exact permissions, no test hooks, no local endpoints, no credential-like strings
Production build loads in Chromium	`production-load.spec.ts`	PASS — manifest, permissions, shortcuts, welcome page
Package	`npm run package`	PASS — prompt-enhancer-0.1.0.zip
Live Groq evaluation	`npm run eval:live`	NOT RUN — skipped (no key in env); api.groq.com blocked (proxy 403)

9.2 Unit test coverage by area

File	Tests	Covers
providers.test.ts	33	Groq request shape, key only in header, error mapping (401/403/404/400/413/429×2/5xx/network/truncated/empty/non-JSON), timeout vs cancel, redaction, retry hints, bounded retry, circuit breaker, user messages
output-validation.test.ts	15	Preserved-item extraction, unchanged detection, blocking vs overridable failures, number separators, language switch
eval-regression.test.ts	14	Dataset coverage + 12 labelled synthetic good/bad rewrites
request-parse.test.ts	13	Nonce delimiting, system rules, no page data, budgets, parser tolerance and rejection
input-and-hint.test.ts	12	Input validation, size cap, secret detection, task hints
settings.test.ts	11	Defaults (Insert anyway off), repair, consent versioning, key storage locations, redaction, masking
frameworks.test.ts	8	Registry completeness and provenance, add-a-technique without engine change, mode filtering, catalog token budget
pipeline.test.ts	7	End-to-end engine with a second (mock) adapter: enhanced, unchanged, blocked, malformed, provider error, no call on invalid input

9.3 Browser tests (36)

- **Editors (each: enhance → verify text → Undo → exact original → no submit → one request):** textarea, input, React-controlled textarea (React state verified), contenteditable, open shadow root, ProseMirror (editor model verified). The first-attempt insertion strategy is asserted per editor.
- **Undo paths:** native Ctrl+Z, restore command, confirmation before overwriting later edits; multi-line vs single-line handling.
- **Refusals (no request sent):** password field, no focused field, closed shadow root, cross-origin iframe.
- **Provider failures (original intact):** 401, daily quota with circuit breaker and override, transient 429 (exactly one retry), 503 (one retry), malformed body, answer-instead-of-rewrite (Insert anyway never offered), dropped details (Copy only by default; opt-in Insert anyway then Undo), unchanged.

- **Request control:** double trigger → one request; Cancel and Esc; typing during request; secret confirmation; missing key / consent opens settings; request body contains no key, page URL or title.
- **Options page:** Groq active, Gemini disabled, masked key, test connection, mode persistence, save/remove key.

9.4 Test validity check

To confirm the browser tests detect real failures, the textarea insertion strategies were deliberately disabled: the textarea and React tests failed while ProseMirror (a different code path) passed; the change was then reverted.

9.5 Planned but not executed

- Live Groq requests (correctness of real responses, JSON-mode reliability per model, latency targets p50 ≤ 3 s / p95 ≤ 8 s).
- Human rubric scoring of live outputs; Groq vs Gemini comparison (Gemini not implemented).
- Manual real-site tests (ChatGPT, Claude.ai, Gemini web, Perplexity, GitHub) and Chrome stable (tests used Playwright's Chromium).
- Toolbar click and keyboard shortcut gestures: automated tests call the same workflow through a test-build hook because automation cannot click the browser toolbar; the shortcut registration itself is verified on the production build.

10. Challenges and decisions

10.1 Technical challenges and solutions

Challenge	Solution
Official docs (Groq, Google, Chrome) and the Groq API unreachable from the build environment	Facts tagged [S]/[A]; runtime model verification; mock provider; optional live harness; honest reporting. No restriction bypassed (D7).
ProseMirror appended pasted text instead of replacing it (stale selection)	Synchronous `selectionchange` after selecting the field; tests now assert the first-attempt strategy so fallbacks cannot hide defects.
Spanish→English translation passed validation (same script)	Stopword-based language guess for en/es/fr/de/pt/it added to the validator.
Prototype toast covered the user's text	Toast anchored above the field, with viewport fallback.
Automation cannot click the toolbar or grant activeTab	Test build adds a hook and localhost host permission; dist check proves neither ships.
Invisible Unicode characters written into source by tooling	Detected by ESLint (no-irregular-whitespace); replaced with explicit escapes.

10.2 Trade-offs

- **Provider:** Groq first for privacy (no free-tier training use documented; ZDR available) and latency; Gemini may follow schemas better — to be measured, not assumed.
- **Free tiers:** limits are not hard-coded; quota errors trip a local circuit breaker to avoid repeated failing requests; one request per enhancement.
- **Permissions vs convenience:** no always-on scripts means no in-page button; the user triggers via toolbar/shortcut.
- **Privacy vs convenience:** consent required before first use; session-only key option; no history.
- **Cost:** BYOK, no backend — zero infrastructure cost; a proxy is only justified for a hosted-key product.
- **Compatibility:** generic verified strategies instead of site-specific hacks; unsupported editors fail safely with Copy.

10.3 Unresolved risks and known limitations

- Real-model behavior (answering instead of rewriting, inventing requirements, JSON reliability) is unmeasured.
- Groq model IDs, limits and error wording are unverified against live documentation.
- Real websites were not tested; site editors change over time.
- API keys in chrome.storage are readable by anyone with access to the browser profile.
- Prompt injection inside the text being rewritten cannot be eliminated by instructions; the structural boundary is plain-text insertion, no execution, no submission, user review.
- `execCommand` is deprecated (still the most compatible path); non-execCommand fallbacks exist.
- The user's Groq key appeared in the conversation transcript and should be rotated.

11. Installation and usage guide

11.1 Build and load

```
npm install
npm run build          # → dist/
npm run package        # optional → prompt-enhancer-0.1.0.zip
```

- Open `chrome://extensions`, enable **Developer mode**, click **Load unpacked**, select `dist/`.
- The settings page opens automatically on first install.

11.2 Configure a Groq key securely

- Create a key at console.groq.com → API Keys. Paste it in settings → **Save key** and accept Chrome's permission prompt for api.groq.com.
- Read the data notice and tick the consent box. Optionally **Test connection** and choose a model.
- On shared machines, untick 'Remember my key' (memory-only storage). Never put keys in source, `.env` files or the repository.

11.3 Use

Action	How
Enhance	Click into a text box, then Alt+Shift+E or the toolbar icon
Undo / restore	Undo in the toast (10 s), Alt+Shift+Z any time on the page, or Ctrl+Z in plain text boxes
Cancel	Cancel in the toast or Esc while enhancing
Change shortcuts	chrome://extensions/shortcuts

11.4 Troubleshooting

Symptom	Fix
Settings open instead of enhancing	Add a key and accept the consent
'Groq rejected your API key'	Create a new key and save it again
'Permission needed for Groq'	Save the key again and accept the permission prompt
'Groq quota reached'	Wait for the reset or use 'Try anyway'
'Model not available'	Choose another model; use Test connection
'This field isn't supported' / 'No text box selected'	Use a regular text box (see supported editors)
'Couldn't update this editor safely'	Text unchanged — use Copy enhanced and paste
Badge shows '!' and nothing else	Page forbids extensions (chrome://, Web Store, PDF viewer)

11.5 Supported editors

Editor	Status
textarea, text/search input, React-controlled textarea, contenteditable, open shadow root, ProseMirror	Tested on local fixtures (automated)
Password/email/tel/number/credit-card fields, closed shadow roots, cross-origin iframes	Refused by design (tested)
Google Docs and canvas editors, Monaco, chrome:// pages	Not supported
Lexical, Slate, Quill, CodeMirror 6	Untested; safe fallback (verify, restore, Copy)
Real sites (ChatGPT, Claude.ai, Gemini, Perplexity, GitHub)	Not yet tested

12. Conclusion and future roadmap

12.1 What was achieved

A working, minimal-permission Manifest V3 extension that captures the focused prompt, enhances it with a task-aware, framework-informed specification through Groq, validates the result, replaces it in place with verified insertion and reliable undo, and never submits. The architecture is provider-neutral, the framework registry is extensible, and 149 automated tests pass. The plan-first workflow, user decisions and every deviation are documented in `plan.md`.

12.2 Remaining limitations

- Not yet exercised against the live Groq API; no measured quality or latency.
- Gemini not implemented.
- Real websites untested; several rich-editor families untested.
- Client-side key storage limits; transcript key exposure requires rotation.

12.3 Prioritized next steps

#	Step	Why
1	Rotate the Groq key; run `GROQ_API_KEY=… npm run eval:live` from a machine with network access	Confirms real behavior, JSON reliability, latency; gives the first real before/after data
2	Human rubric scoring; tune instructions (bump INSTRUCTIONS_VERSION) and default model	Quality evidence (plan §20.3 thresholds)
3	Manual tests on ChatGPT, Claude.ai, Gemini web, Perplexity, GitHub	Real compatibility matrix
4	Verify Gemini terms/data use/region rules, then implement the Gemini adapter (M5)	Second provider and comparison
5	Fixtures for Lexical, Slate, Quill, CodeMirror 6	Broader editor coverage
6	Optional UX: diff view, per-use mode picker, opt-in history with retention rules	Post-MVP features
7	Web Store readiness: privacy policy, listing disclosures, permission justifications, BYOK vs proxy decision	Possible publication

12.4 Deliverable locations

- Extension build: `dist/` (load unpacked) and `prompt-enhancer-0.1.0.zip` (both generated; not committed).
- Documentation: `README.md`, `plan.md`; this report: `project_report.pdf`.
- Report sources: `report/` (generator `build_report.py`, verbatim prompts, test outputs, screenshots, diagrams).